



# 南京大學

## 本科畢業論文

院 系 计算机学院

专 业 计算机科学与技术

题 目 手机 APP 自动化测试

年 级 2022 学 号 225220003

学生姓名 林能宇

指导教师 聂长海 职 称 教授

提交日期 2026 年 6 月 9 日



# 南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：手机 APP 自动化测试）是在指导教师的指导下严格按照学校和学院有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：林能宇

学号：22522003

日期：2026年6月9日

# 南京大学本科生毕业论文（设计、作品）中文摘要

**题目：**手机 APP 自动化测试

**院系：**计算机学院

**专业：**计算机科学与技术

**本科生姓名：**林能宇

**指导教师（姓名、职称）：**聂长海 教授

**摘要：**

移动互联网的普及速度越来越快，智能手机 APP 已经深度融入到公众日常生产生活的各个场景里，跟着应用市场上架的程序规模不断变大，软件测试环节对应用运行质量的支撑作用越来越明显，但移动终端硬件型号太多太杂，不同版本操作系统的兼容性差别很大，再加上测试需要覆盖的场景复杂多变，靠人工完成的传统测试方式，已经跟不上现在对质量保障效率和可靠性的高要求了。面对这种情况，搭建一套成体系、方便迭代的移动端 APP 自动化测试框架，已经是移动应用测试领域迫切需要突破的主要问题。

本文围绕移动端 APP 自动化测试展开，利用 Appium 与 Selenium WebDriver 作为核心技术支持，融合了模块化设计理念、Page Object 设计模式、数据驱动测试思路以及日志报告生成机制，开发出一套适配 Android 平台的移动应用自动化测试框架。这套框架采用五层架构设计：测试用例管理层负责测试用例的整体安排和调度工作，支持参数化配置以及数据驱动的自动执行；页面元素封装层依靠 Page Object 模式，把界面元素的定位逻辑和操作方法封装成可以独立调用的对象，大幅降低了测试脚本和前端界面更新之间的耦合关联，公共方法层整合了控件等待、异常捕获、运行截图、驱动初始化这类通用功能，给测试脚本稳定运行给出基础支持，配置管理层把测试环境的各项参数做统一管理，能做到不同终端设备和系统版本之间快速切换适配，日志与报告模块可以自动保存测试全流程的数据，还能输出可视化的分析报告，给测试结果复盘和体系迭代优化提供依据。

本研究选了 Appium 官方给出的示例应用 ApiDemos 6.0.10 当作主要测试对象，分别在安装了 Android 16.0 系统的模拟器，还有安装了 Android 13.0 系统的实体设备上，运行包含登录操作、表单录入、页面切换、UI 元素交互在内的多

种典型测试用例。同时，本文还补充使用 SauceLabs My Demo App 作为第二个测试对象，对购物流程、登录和 Website 搜索等场景进行测试，让测试内容不局限在单一示例应用，另外，本文还加入了基于 ADB Monkey 的随机稳定性测试，在 Android 16.0 模拟器中向 ApiDemos 发送大量随机事件，通过日志观察应用不会不会出现崩溃、无响应或异常退出等问题。

从测试数据能看出，这次设计出来的自动化测试框架，可以在不同硬件终端和不同系统版本的环境里稳定运行，大部分测试用例都能跑出我们想要的结果，不管是应对动态界面布局变化，还是适配多种终端设备，这个框架都有着不错的适配能力和可靠性。Monkey 测试的结果也表明，随机稳定性测试可以覆盖到固定流程测试不容易碰到的页面和操作路径，还能在一定程度上找出潜在的稳定性风险。

本研究做出来的实践结果能够说明，基于 Appium 和 Selenium 搭建出来的分层式自动化测试框架，确实可以提高移动应用测试的执行速度，同时还能增强测试脚本的复用性和可迭代性。在结合 ADB Monkey 随机测试之后，整套测试方法变得更加完整，也能给软件开发项目中推进移动应用自动化测试工作，给出有参考价值的设计思路和落地方向。

**关键词：**手机 APP 自动化测试；Appium；Selenium；Page Object 模式；ADB Monkey；Android

# 南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Mobile App Automated Testing

DEPARTMENT: School of Computer Science

SPECIALIZATION: Computer Science and Technology

UNDERGRADUATE: Lin Neng Yu

MENTOR: Professor Changhai Nie

ABSTRACT:

As mobile internet grows really fast these days, smartphone apps have spread into almost every part of people's daily life. As more and more apps get released, testing new software has become more important than ever to make sure an app works well. The big range of different mobile phone models, different versions of operating systems, plus all the complicated ways people actually use apps, makes old-style manual testing not work very fast, and it can't keep up with what we need now for reliability and speed. Because of this, people who do mobile app testing really need a good organized automation testing framework that is also easy to keep updated.

This study looks at automated testing for mobile apps, and uses Appium and Selenium WebDriver as main technical support. It brings together modular design rules, the Page Object design pattern, data-driven testing plans, and also includes automatic logging and reporting systems. We built a layered testing framework that works on the Android platform. This framework is made up of five different layers. The first one is the Test Case Management Layer, it organizes and arranges test cases, and supports parameterized and data-driven running of tests. The next is the Page Element Encapsulation Layer, which puts element finding rules and operation methods into separate objects through the Page Object pattern, this helps lower the connection between test scripts and changes in the UI. Then there is the Common Methods Layer, it gives all the general functions like waiting for controls, handling exceptions, taking screenshots and starting the driver. After that is the Configuration Management Layer, which keeps track of test environment parameters and lets the framework adjust quickly when used on different devices or different OS versions. The last layer is Logging and Reporting

Layer, it automatically keeps a full record of the whole testing process and makes visual analysis reports, which helps people check results and make improvements step by step.

For this study, we picked Appium's official sample app, ApiDemos 6.0.10, to be our main test subject. We ran all tests on two devices: an Android 16.0 emulator and a real Android 13.0 phone. The tests included the most common kinds of use cases, such as logging in, filling out forms, moving between pages, and interacting with different UI elements. We also added SauceLabs My Demo App as a second test object. It let us test more scenarios, like full shopping processes, logging in, and searching on a website, so our testing didn't just stay limited to one single sample app. We also did ADB Monkey random stability testing on the Android 16.0 emulator. This step sent many random events to the ApiDemos app, and we checked the logs to look for crashes, ANR issues, or any times the app quit abnormally.

The test results show that the automation framework we designed works stable across different types of hardware and operating system environments, and most test cases run just like we expected. The framework show good adaptability and reliability when it deals with dynamic UI layouts and many different device configurations. The Monkey testing also helps prove that this setup can reach pages and operations that fixed workflow tests might miss, and it finds possible stability risks that only show up when random operations happen.

The test results we got from practice show that the layered automation testing framework built with Appium and Selenium can improve testing efficiency a lot, and it also makes scripts easier to reuse and keep. Adding ADB Monkey random testing fills out the whole set of testing methods we use, and it gives useful reference info for people who want to do mobile App automation testing in their own software development projects.

**KEYWORDS:** Mobile App Automation Testing; Appium; Selenium; Page Object pattern; ADB Monkey; Android

# 目 录

|                          |    |
|--------------------------|----|
| 目 录                      | VI |
| 第一章 导论                   | 1  |
| 1.1 研究背景及意义              | 1  |
| 1.1.1 研究背景               | 1  |
| 1.1.2 研究意义               | 1  |
| 1.2 研究现状                 | 2  |
| 1.2.1 国外研究现状             | 2  |
| 1.2.2 国内研究现状             | 3  |
| 1.2.3 研究现状评述             | 3  |
| 1.3 本文主要研究内容             | 4  |
| 1.4 论文组织结构               | 4  |
| 第二章 相关技术与理论基础            | 6  |
| 2.1 手机 APP 测试            | 6  |
| 2.1.1 手机 APP 测试的概念       | 6  |
| 2.1.2 手机 APP 测试类型        | 7  |
| 2.2 自动化测试                | 7  |
| 2.2.1 自动化测试的优势           | 8  |
| 2.2.2 自动化测试的局限性          | 9  |
| 2.3 Selenium 自动化测试技术     | 10 |
| 2.4 Appium 自动化测试工具       | 10 |
| 2.5 Android 测试环境相关技术     | 12 |
| 2.5.1 Android Studio 模拟器 | 12 |
| 2.5.2 真机测试与模拟器测试         | 12 |

|            |                            |           |
|------------|----------------------------|-----------|
| 2.5.3      | ADB Monkey 随机测试工具          | 13        |
| 2.6        | 自动化测试框架设计相关理论              | 14        |
| 2.6.1      | 模块化设计思想                    | 14        |
| 2.6.2      | Page Object 设计模式           | 14        |
| 2.6.3      | 数据驱动测试                     | 15        |
| 2.6.4      | 日志记录与测试报告                  | 15        |
| 2.7        | 本章小结                       | 15        |
| <b>第三章</b> | <b>手机 APP 自动化测试需求分析</b>    | <b>17</b> |
| 3.1        | 测试对象                       | 17        |
| 3.2        | 测试环境                       | 17        |
| 3.3        | 测试需求                       | 18        |
| 3.3.1      | 功能测试需求                     | 18        |
| 3.3.2      | UI 测试需求                    | 19        |
| 3.3.3      | 兼容性与性能需求                   | 19        |
| 3.4        | 测试用例设计                     | 19        |
| 3.5        | 本章小结                       | 21        |
| <b>第四章</b> | <b>手机 APP 自动化测试框架设计</b>    | <b>23</b> |
| 4.1        | 测试用例管理层                    | 24        |
| 4.2        | 页面元素封装层 (Page Object 模式)   | 24        |
| 4.3        | 公共方法层                      | 25        |
| 4.4        | 配置管理层                      | 26        |
| 4.5        | 日志与报告模块                    | 27        |
| 4.6        | 本章小结                       | 28        |
| <b>第五章</b> | <b>手机 APP 自动化测试实现与结果分析</b> | <b>29</b> |
| 5.1        | 测试环境搭建                     | 29        |
| 5.2        | 自动化测试脚本实现                  | 30        |
| 5.3        | 典型测试用例执行                   | 31        |
| 5.3.1      | 登陆测试                       | 32        |



|                               |                                 |           |
|-------------------------------|---------------------------------|-----------|
| 5.3.2                         | 表单以及元素可见性测试 . . . . .           | 34        |
| 5.3.3                         | 结合综合测试 . . . . .                | 37        |
| 5.3.4                         | SauceLabs My Demo App . . . . . | 41        |
| 5.3.5                         | Monkey 随机稳定性测试 . . . . .        | 43        |
| 5.4                           | 测试结果分析 . . . . .                | 45        |
| 5.4.1                         | 综合测试结果分析 . . . . .              | 45        |
| 5.4.2                         | 综合测试结果分析 . . . . .              | 47        |
| 5.4.3                         | 手动测试与自动化测试比较 . . . . .          | 49        |
| 5.4.4                         | Monkey 测试结果分析 . . . . .         | 51        |
| 5.5                           | 本章小结 . . . . .                  | 53        |
| <b>第六章 总结与展望 . . . . .</b>    |                                 | <b>56</b> |
| 6.1                           | 研究总结 . . . . .                  | 56        |
| 6.2                           | 研究不足 . . . . .                  | 57        |
| 6.3                           | 后续改进与展望 . . . . .               | 58        |
| <b>参考文献 . . . . .</b>         |                                 | <b>60</b> |
| <b>致 谢 . . . . .</b>          |                                 | <b>63</b> |
| <b>附录 A 测试报告明细 . . . . .</b>  |                                 | <b>64</b> |
| <b>附录 B 本文代码存储库 . . . . .</b> |                                 | <b>66</b> |

# 第一章 导论

## 1.1 研究背景及意义

### 1.1.1 研究背景

最近这些年，移动智能终端应用行业一直保持高速发展，跟着移动设备的用户数量越来越多，市场对高质量应用程序的需求也一直在涨。智能手机做应用开发的时候，得从好几个维度保证产品质量，具体包括核心功能与可靠性（Core Functionality and Reliability）、直觉化用户体验（User Experience, UX）、界面交互设计（User Interface, UI）、安全与隐私保护（Security and Privacy），还有可拓展性与运维便利性（Scalability and Maintenance）等多个方面。

这样的背景下，应用程序测试（Software Testing）已经变成软件开发流程里不能缺少的关键一步。测试工作主要的目标就是保证应用程序跑起来稳定，确实能对上用户实际用的时候的需求，现在智能移动终端种类太杂，不同的操作系统版本之间差别很明显，应用程序通常得覆盖好多种设备环境做完测试，测试载体分实体真机和虚拟模拟设备两种。

纯人工一步步走完测试流程的做法，不光要花掉超多时间和人力成本，碰到多设备适配、多场景覆盖的测试要求时，效率会显著降低，最后拖慢整个软件开发的推进速度。引入自动化测试（Automation Testing），刚好能解决这个麻烦，是提高测试效率、保证执行效果统一的可行办法。

### 1.1.2 研究意义

现在移动应用自动化测试这块已经有不少成熟工具了，比较有代表性的有 Appium、Maestro、KaneAI 还有 Katalon 这些。本次研究选了 Appium 作为移动应用自动化测试的核心支撑框架，同时配上 Selenium 当辅助工具，一起帮移动应用自动化测试流程落地执行。

虽然现在已经有挺多种自动化测试工具了，但实际做移动应用测试落地的时候，推自动化测试方案还是有好多问题等着解决，其中最突出的问题就是，如何搭出一套架构清楚、适配性好，还方便后期维护的自动化测试框架。没有科学合理的框架设计，测试脚本的管理和拓展都会受阻，一旦被测应用的界面交互或者核心功能改了，原来写好的测试脚本很容易就没法正常运行了。

本次研究以手机 APP 测试作为核心研究对象，基于 Appium、Selenium 这些已经成熟的自动化测试技术，找一找移动应用自动化测试体系的设计和落地方法。搭出逻辑清楚的自动化测试流程和架构体系后，优化测试执行的效率，提高测试脚本的复用能力和可维护性，希望能给行业里做移动应用测试的实际工作给一些可以参考的思路。

本次研究在软件测试领域同时有实践参考价值和理论探索的意义，搭好结构化的自动化测试框架，能进一步提高移动应用测试流程的运行效率和稳定性，加强测试脚本后期维护的能力。研究结果可以给软件开发从业者、相关领域研究人员做移动应用自动化测试实践当参考，尤其是对靠着 Appium 和 Selenium 搭自动化测试框架的场景，有直接的借鉴用处。

## 1.2 研究现状

### 1.2.1 国外研究现状

近年来，国外学者对移动应用自动化测试展开了大量研究。Kong 等人<sup>[1]</sup>对 Android 应用自动化测试方法进行了系统性综述，分析了当前移动应用自动化测试工具的发展现状及其面临的挑战。Kirubakaran 等人<sup>[2]</sup>指出，由于移动设备硬件配置、操作系统版本以及屏幕尺寸存在差异，移动应用测试相比传统软件测试具有更高的复杂性，因此需要借助自动化测试技术来提高测试效率。

此外，一些研究还关注自动化测试框架的设计问题。Tao 和 Gao<sup>[3]</sup>提出了一种基于 GUI Ripping 技术的移动应用自动化测试方法，通过自动识别界面元素来生成测试用例，从而提高测试覆盖率。Prathibhan 等人<sup>[4]</sup>则提出了一种基于云环境的 Android 应用自动化测试框架，以支持大规模设备测试。

Selenium 传统上以 Web 自动化而闻名，现在已扩展到移动应用程序领域。Sinclair<sup>[5]</sup>指出，Selenium+Appium 能够支持跨平台移动应用测试，并能够与多种

测试框架进行集成，因此在移动应用自动化测试中得到了广泛应用。

### 1.2.2 国内研究现状

最近这些年，国内关于移动应用自动化测试技术的相关研究一直都在变多，张佳慧<sup>[6]</sup>把研究重点放在 Appium 架构下的移动设备测试技术上，还做了系统性分析，结合实际的测试需求搭出了一套完整的 Appium 自动化测试落地方案，有效缩短了移动应用测试的整体周期，还提高了测试的执行效率，黄冬丽等学者<sup>[7]</sup>把 Selenium 和 Appium 工具结合在一起，针对多端登录场景的测试需求做了相关研究，梳理并且重构了标准化的测试执行流程之后，顺利完成了多设备适配环境下的自动化测试覆盖工作。李佳<sup>[8]</sup>选手机测速类应用当作研究的切入点，整理了这类应用在上传速率、下载速率还有 Ping 值检测这些网络状态测试环节的具体应用逻辑，给移动应用测试场景的边界拓展给出了可行的参考方向。

整理现有的研究成果能发现，目前大部分探索还是围绕单一平台或者特定业务场景的测试落地来进行，针对跨平台适配、多版本兼容、复杂网络环境耦合的综合性测试框架，相关研究还存在比较明显的缺口。虽说国内外学界在移动应用自动化测试领域已经攒下了不少突破性的成果，但怎么搭建出架构分层清楚、扩展能力强而且运维成本比较低的自动化测试框架，依旧是这个领域需要解决的主要共性问题。

### 1.2.3 研究现状评述

综上所述，国内外学者在移动应用自动化测试领域均取得了显著进展。国际研究侧重于测试方法和工具的创新以及大规模设备兼容性，而国内研究则更侧重于实际应用场景的自动化实现和测试效率的提升。然而，设计一个结构清晰、灵活且易于维护的自动化测试框架仍然是当前移动应用自动化测试研究和实践中需要重点关注的问题。因此，本文以 Appium 和 Selenium 为基础，探索移动应用自动化测试框架的设计方法，旨在提高测试效率、增强脚本复用性并提升移动应用测试的整体质量。

### 1.3 本文主要研究内容

本文围绕手机 APP 自动化测试技术展开研究，主要研究内容包括以下几个方面。

- 一、 分析手机 APP 测试具备什么样的特点，又有哪些实际需求。结合移动应用  
在功能、界面、兼容性和稳定性几个方面的测试要求，梳理出手机 APP 测试  
过程里存在的主要问题。
- 二、 研究 Appium 和 Selenium 在自动化测试当中分别是怎么应用的。通过拆解  
分析 Appium 的工作原理、元素定位的具体方式，还有 Selenium WebDriver  
的相关运行机制，明确这篇文章做自动化测试要用到的技术基础。
- 三、 设计手机 APP 的自动化测试框架。本文从测试用例管理、页面对象封装、  
公共方法调用、配置管理、日志记录还有测试输出报告等多个角度做了框  
架设计，用来提高测试脚本的复用性和可维护性。
- 四、 完成典型测试场景的自动化功能实现，并且对测试结果做分析。挑选登录、  
表单输入、页面跳转还有 UI 校验这类比较典型的功能场景，编写对应的自  
动化测试脚本，之后对测试执行出来的结果做整理和分析。

### 1.4 论文组织结构

本文共分为六章，各章内容安排如下。

**第一章**为导论，主要介绍本文的研究背景、研究意义、国内外研究现状、主要研究内容以及论文结构安排。

**第二章**为相关技术与理论基础，主要介绍手机 APP 测试、自动化测试、Appium、Selenium、Android Studio 以及自动化测试框架等相关技术。

**第三章**为手机 APP 自动化测试需求分析，主要分析测试对象、测试环境、功能测试需求、UI 测试需求和兼容性测试需求，并设计相应的测试用例。

**第四章**为手机 APP 自动化测试框架设计，主要从总体架构、测试用例层、页面元素封装层、公共方法层、配置管理层、日志与报告模块等方面进行设计。

**第五章**为手机 APP 自动化测试实现与结果分析，主要介绍测试环境搭建、自动化脚本实现、典型测试用例执行过程以及测试结果分析。

**第六章**为总结与展望，主要总结本文完成的研究工作，分析存在的不足，并对后

续改进方向进行展望。

## 第二章 相关技术与理论基础

### 2.1 手机 APP 测试

#### 2.1.1 手机 APP 测试的概念

手机 APP 测试 (Mobile Application Testing)<sup>[9]</sup> 是软件开发过程中的一个重要环节, 主要用于保证移动应用程序在不同设备和平台上的质量、功能性、可用性、安全性和性能。与桌面应用程序或 Web 应用程序不同, 手机 APP 的运行环境更加复杂, 容易受到设备型号、操作系统版本、屏幕尺寸等因素的影响。具体对比如下表所示:

表 2-1 手机 APP 测试与桌面/Web 应用测试的比较

| 参数      | 手机 APP 测试                              | 桌面/Web 应用测试                                          |
|---------|----------------------------------------|------------------------------------------------------|
| UI      | 手机 APP 通常会使用触控手势、传感器以及移动端特有功能。         | 更多依赖键盘、鼠标等传统输入设备。                                    |
| 平台      | 手机 APP 需要在多种平台上进行测试, 例如 iOS 和 Android。 | 桌面应用通常面向特定操作系统, 硬件规格差异相对较小。                          |
| 网络环境    | 手机 APP 需要测试其不同网络环境下的可靠性、性能以及离线使用能力。    | 虽然基于 Web 的桌面程序也可能需要在不同网络环境下测试, 但桌面应用通常运行在相对稳定的网络环境中。 |
| 测试工具和框架 | 为了在真实、模拟器上进行测试, 移动端测试通常需要专门的工具和框架      | 桌面测试工具可能更注重自动化、不同操作系统之间的兼容性以及与开发环境的集成。               |

手机 APP 测试通常包括一系列测试活动, 目的是在应用发布给最终用户之前, 发现并修复应用中可能存在的缺陷或问题。常见的手机 APP 测试方式包括

人工测试、Beta 测试、设备测试以及自动化测试等。本文主要关注手机 APP 的自动化测试，即通过自动化测试工具和测试脚本，提高移动应用测试的执行效率和可维护性。

### 2.1.2 手机 APP 测试类型

根据具体的测试目标和测试内容，手机 APP 测试可以划分为多种类型。本文主要关注与自动化测试关系较为密切的几类测试，包括：

- 1) 功能测试 (Functional Testing)：功能测试主要用于验证软件程序是否满足用户需求和预期，检查应用的各项功能以及交互流程是否能够按照设计要求正常运行。
- 2) 性能与稳定性测试 (Performance and Stability Testing)：性能与稳定性测试主要用于评估应用在常规场景和高强度场景下的响应速度、运行效率和稳定性，检查应用是否能够保持较好的运行状态。
- 3) 兼容性测试 (Compatibility Testing)：兼容性测试主要用于确认应用程序是否能够在不同平台、设备和配置环境下正常运行，并为不同用户提供相对一致的使用体验。
- 4) 安全性测试 (Security Testing)：安全性测试主要用于发现并修复应用中可能存在的安全漏洞，保护用户信息，防止非法访问，并保证应用数据的完整性和保密性。
- 5) 用户界面测试 (User Interface Testing)：用户界面测试不仅关注页面的视觉呈现效果，还关注用户是否能够顺利地在应用中完成操作任务，避免在使用过程中遇到界面显示或交互障碍。

在实际测试过程中，功能测试和用户界面测试通常更适合进行自动化实现；性能、兼容性和稳定性测试可以借助自动化工具进行辅助验证；而安全性测试一般需要结合专门的安全测试方法和工具进行分析。

## 2.2 自动化测试

自动化测试 (Automation Testing)<sup>[10]</sup>是一种利用特定测试工具、测试脚本和测试框架来代替部分人工操作的软件测试方法。与人工测试相比，自动化测试能



够更快速、更准确地执行测试任务，尤其适用于一些重复性较高、流程较固定的测试场景。人工测试更适合探索性测试、可用性测试、用户体验测试、视觉验证测试以及临时性测试等需要人工判断的场景。而自动化测试则更适合重复性测试、数据驱动测试、核心业务测试、结果明确的测试，以及耗时较长、操作较繁琐的测试任务。也就是说，自动化测试并不意味着完全取代人工测试。在实际软件测试过程中，人工测试和自动化测试各有适用范围，二者通常根据不同的测试目标进行配合使用。

随着移动应用测试技术的发展，越来越多的手机 APP 自动化测试工具被应用于实际测试过程中，例如 BrowserStack<sup>[11]</sup>、LambdaTest TestMu AI<sup>[12]</sup>、Robotium 和 Appium<sup>[13]</sup>等。这些工具可以帮助测试人员在不同设备和不同环境下更高效地执行手机 APP 测试用例。其中，Appium 是一种应用较为广泛的开源移动端自动化测试框架，支持 Android 和 iOS 平台。因此，本文将以 Appium 作为核心自动化测试工具，围绕基于 Appium 的手机 APP 自动化测试设计与实现展开研究。

### 2.2.1 自动化测试的优势

自动化测试在软件测试中具有多方面优势，尤其在手机 APP 测试中表现较为明显。

- 1) 执行速度较快：自动化测试的执行速度通常快于人工测试，尤其适用于重复性较高的测试用例。通过自动执行测试脚本，可以减少人工重复操作所花费的时间。
- 2) 测试准确性较高：自动化测试可以减少人为操作失误，并按照预先设定的步骤执行测试，从而保证每次测试过程具有较好的一致性。
- 3) 测试覆盖范围更广：自动化测试可以在多个测试环境和多组测试数据下执行大量测试用例，有助于扩大测试范围，提高测试覆盖率。
- 4) 支持脚本复用：自动化测试脚本可以在应用的不同版本或不同构建中重复使用。对于功能变化较小的测试场景，复用测试脚本能够减少重复编写测试用例的工作量。
- 5) 适合回归测试：在应用功能更新或代码修改之后，需要验证原有功能是否仍然正常运行。自动化测试可以快速执行已有测试用例，从而提高回归测

试的效率。

- 6) 支持 CI/CD 集成：自动化测试可以与持续集成和持续交付流程结合，在 DevOps 流水线中实现持续测试，从而提高软件交付效率。
- 7) 便于结果记录与持续测试：自动化测试可以记录测试结果和失败信息，并支持测试脚本重复执行，便于后续分析和持续验证。

### 2.2.2 自动化测试的局限性

尽管自动化测试具有较强的实用性，但在实际应用过程中仍然存在一定的局限性<sup>[14][15]</sup>

- 1) 初始成本较高：自动化测试通常需要测试工具、测试环境、测试设备，有时还需要具备相关经验的测试人员。自动化测试框架的搭建以及首批测试脚本的编写也需要一定时间。因此，对于小型项目或小型团队而言，前期投入成本可能相对较高。
- 2) 脚本维护成本比较高：自动化测试脚本大多和被测应用绑定得比较深。只要应用的界面布局、业务流转逻辑或者元素定位规则改了，现有的测试脚本大概率没法正常跑起来，这就要求测试脚本得一直迭代优化、更新状态，最后会让后续维护阶段投入的人力变多。
- 3) 对做测试的人能力要求更高：自动化测试不是说只会操作测试工具就能做好的，做自动化测试的人还得懂系统的测试方法、会基础编程、了解常用测试框架的原理，还要知道被测系统的主要运行逻辑。要是测试团队没攒够对应的技术本事，一般很难搭出好用、还能长期维护的自动化测试体系。
- 4) 适配场景有明显的局限：有不少测试场景很难改成自动化，尤其是和用户主观感受、视觉效果检查、复杂多路径交互有关的测试需求。因为自动化脚本一般都是跟着提前设好的固定步骤一步步跑，想要复现真实用户随机的复杂操作，本身就有一定的能力不足。
- 5) 测试结果有可能出现误判：自动化测试跑的时候，偶尔会出现假失败或者假通过的异常情况。比如测试用例执行时报错，可能是网络速度太慢、终端设备反应慢、测试环境运行不稳这些原因导致的，不一定就是应用本身有功能问题，这类异常会给问题排查和脚本调试多添不少工作量。

- 6) 要体现价值得攒时间、还要有清楚的规划：自动化测试的效果一般得推进一段时间才能完全显现，项目刚开始的时候，测试团队通常要投很多资源做环境搭建、选框架、写脚本这些工作。要是没定好明确的自动化测试推进计划，最后做出来的效果大概率 and 预期差得比较多。

自动化测试确实能明显提高测试环节的运行效率，但要落地得有对应的资金、时间、维护资源和技术能力支撑才行。在手机 APP 的测试工作里，应该先把自动化测试技术用在适配性好、重复度高、流程规则清楚稳定的测试场景里，别脱离实际需求盲目把所有测试都改成自动化。

## 2.3 Selenium 自动化测试技术

Selenium<sup>[16][17]</sup>是现在用得非常多的开源自动化测试工具，主要用来给 Web 应用做功能和交互测试，这个工具自带 Selenium WebDriver 组件，能通过客户端给目标浏览器发操作指令，完成网页元素的交互控制和状态检查。WebDriver 可以高度模拟用户在浏览器里的各种操作，包括点按钮、输入文字、选下拉菜单、提取页面信息等等，靠这些操作完成网页功能逻辑和界面交互流程的自动验证。

Selenium 和 Appium 底层的运行逻辑是相通的。Appium 做移动端自动化测试的时候，也用了 WebDriver 协议，测试脚本通过符合 WebDriver 规范的接口和 Appium Server 完成数据交换，再由服务端把具体的操作指令传给移动端驱动程序去执行。由于 Selenium 已经成熟并且易于理解，测试人员在学习和使用 Appium 时，可以借助 Selenium 的知识快速掌握 WebDriver 的基本用法，从而提高移动端自动化测试脚本编写效率。

在本文中，Selenium 主要作为 Appium 的技术基础和辅助工具使用<sup>[5]</sup>。通过理解 Selenium 的工作机制和操作方式，可以帮助测试人员更好地编写和管理 Appium 自动化测试脚本，同时在后续自动化测试框架设计中，Selenium 提供了可复用的测试思路和 WebDriver 标准接口，支持移动 APP 功能测试和界面验证。

## 2.4 Appium 自动化测试工具

Appium<sup>[13]</sup>是一种开源的移动端自动化测试工具，主要用于 Android 和 iOS 应用程序的自动化测试。它支持原生应用、混合应用以及移动 Web 应用测试，并

且可以使用 Java、Python、JavaScript 等多种编程语言编写测试脚本。与部分平台专用测试工具相比，Appium 具有较好的跨平台能力，能够在不同移动操作系统和设备环境下执行自动化测试任务。

在手机 APP 自动化测试中，Appium 可以模拟用户对应用的常见操作，例如点击按钮、输入文本、滑动页面、切换页面以及验证界面元素等。测试人员可以通过编写测试脚本控制移动设备或模拟器，从而完成对应用功能和界面交互流程的验证。由于 Appium 与 Selenium WebDriver 在协议和使用方式上具有一定联系，因此它也便于测试人员理解和使用 WebDriver 风格的自动化测试方法。

操作自动化测试的时候，定位界面里的元素是完成操作的关键一步。Appium 给出了多种元素定位的方法，常用的有：

- **id 定位：**利用元素的 resource-id 来查找，定位速度快，不容易出问题
- **Accessibility ID 定位：**借助可访问性标识来查找，适合跨平台操作，较好的稳定性。
- **Xpath 定位：**按照元素在页面里的路径完成定位，灵活度很高，但如果页面结构发生改变，就很容易失效。
- **class name 定位：**通过控件类型来查找，适合找同一类控件，若同类元素比较多，就得搭配索引或者其他条件来用。
- **text 定位：**借助元素显示出来的文本内容查找，适合文字固定的场景，但是在多语言环境里可能不太稳定。

结合 Appium 的工作原理还有各种元素定位方式，测试人员可以模仿用户在手机 APP 里的常见操作，比如点按钮、输入文字、滑动页面、跳转页面、检查界面元素这些。这让 Appium 可以给应用的功能和界面流程做自动化验证，给之后搭建结构化的自动化测试框架打下可靠基础，这些操作也方便记录测试结果和做持续测试，应用更新之后测试脚本还能重复运行，用来快速检查核心功能是不是正常运行，从而提高测试效率还有测试反馈的能力。

## 2.5 Android 测试环境相关技术

### 2.5.1 Android Studio 模拟器

Android Studio 模拟器是 Android 官方提供的虚拟设备环境，用于在电脑上模拟 Android 手机或平板的运行环境。通过模拟器，测试人员可以在不同 Android 版本、屏幕尺寸和分辨率下执行测试，而不需要每次都使用真实设备。模拟器支持对应用的安装、卸载、启动、页面操作等基本功能，同时可以模拟网络状况、地理位置、传感器和硬件条件变化。对于移动 APP 自动化测试来说，模拟器能够快速搭建测试环境、调试脚本和验证功能，是进行初步测试和框架开发的重要工具。

### 2.5.2 真机测试与模拟器测试

在移动端测试中，使用不同设备测试 App 或 Web 对于保证性能、可用性和兼容性非常重要。移动测试主要有三种方式：模拟器（Emulator）、仿真器（Simulator）和真实设备（Real Device）<sup>[18]</sup>。本文将重点关注模拟器（Emulator）和真实设备（Real Device）两种方法。

表 2-2 模拟器与真实设备测试的区别

| 参数           | 模拟器（Emulator）             | 真实设备（Real Device）                      |
|--------------|---------------------------|----------------------------------------|
| 性能测试         | 无法准确测量 CPU 使用率、电池消耗或内存占用。 | 能够在真实使用环境下提供准确的性能指标。                   |
| 硬件相关测试       | 可以模拟部分硬件功能，但无法覆盖所有硬件特性。   | 对于依赖硬件的功能（如传感器、摄像头等），真实设备测试是必不可少的。     |
| 网络与连接测试      | 提供有限或模拟的网络环境。             | 可以在实际 Wi-Fi、移动网络下测试应用，并评估真实网络延迟对应用的影响。 |
| App 对应不同的 OS | 能够快速在多个操作系统版本下测试应用。       | 可以展示应用在真实系统更新、自定义设置和厂商定制环境下的实际运行效果。    |

综上，模拟器适合快速测试、调试和验证功能，而真实设备能够更接近用户实际使用环境，用于性能评估、硬件功能测试和网络兼容性测试。实际测试中，通常建议在开发初期使用模拟器进行功能验证，脚本稳定后再在真实设备上回归和多设备兼容性测试，以保证自动化测试结果的可靠性和适用性。在掌握了 Appium、Selenium 以及 Android 测试环境的基础上，下一节将介绍自动化测试框架设计的相关理论，为后续具体框架的搭建提供参考。

### 2.5.3 ADB Monkey 随机测试工具

ADB Monkey 是 Android SDK 给出的一款命令行随机测试工具，一般用来给 Android 应用做稳定性以及压力测试。跟 Appium 这类按照提前写好的脚本执行操作的自动化测试工具不一样，Monkey 测试的特点是给指定应用随机发送大量用户事件，比方说点击、滑动、按键、切换页面这类操作，测试过程里会观察应用会不会出现崩溃、无响应（ANR）或是异常退出这类问题，所以，Monkey 测试更适合找出应用在非预期操作和随机交互场景下的稳定性问题。

移动 APP 测试里，用户实际操作一般都带点随机性，虽然 Appium 自动化脚本可以按着固定流程检查登录、填表单、跳转页面、控件交互这些功能是否符合预期，但它覆盖的主要是提前设计好的测试路径，对用户随机点击、快速切换页面或是连续滑动这类非固定操作场景的覆盖相对比较有限。ADB Monkey 测试能补上这块不足，利用大量随机事件模拟用户不确定的操作，以此检验应用在复杂交互环境里的容错能力还有运行稳定性。

ADB Monkey 测试借助 ADB 命令来运行，它的基本格式如下。

**adb shell monkey -p [应用包名] -v [随机事件数量]**

其中，‘-p’ 用于指定被测应用的包名，避免随机事件作用到其他应用；‘-v’ 用于输出较详细的测试日志；最后的数字表示随机事件数量。为了使测试过程更加稳定，也可以通过 ‘-throttle’ 参数设置事件之间的间隔时间，例如：

**adb shell monkey -p io.appium.android.apis -throttle 100 -v 1000**

该命令表示对包名为 ‘io.appium.android.apis’ 的应用发送 1000 个随机事件，并将每个事件之间的间隔设置为 100 毫秒。测试人员可以根据测试目标调整事件数量和事件间隔。如果需要保存测试日志，也可以将命令输出重定向到日志文件中，便于后续分析崩溃信息、异常信息和测试执行过程。

在本文研究中，ADB Monkey 测试作为 Appium 自动化测试的补充方法使用。Appium 主要用于验证固定业务流程和界面元素交互的正确性，而 Monkey 测试主要用于验证应用在随机操作和异常交互情况下的稳定性。二者结合后，可以使测试方法更加完整：一方面通过 Appium 保证核心功能流程能够按照预期执行，另一方面通过 Monkey 测试观察应用在随机事件冲击下是否保持稳定运行，从而提高手机 APP 自动化测试结果的可靠性和覆盖范围。

## 2.6 自动化测试框架设计相关理论

在手机 APP 自动化测试中，构建一个结构清晰、易维护、可扩展的自动化测试框架非常重要。一个好的框架不仅提高测试效率，还能增强脚本的复用性和可维护性。本节介绍几种在自动化测试框架设计中常用的理论和方法，为后续具体框架搭建提供参考。

### 2.6.1 模块化设计思想

模块化设计是将复杂测试流程拆分为多个独立功能模块，每个模块完成特定任务。在手机 APP 自动化测试中，可以将测试用例、页面操作、公共方法、配置管理等分成不同模块。优势和应用：

- 模块独立，低耦合、高内聚，修改某个模块不会影响其他模块。
- 便于维护和扩展，例如应用界面更新时只需要修改相关模块。
- 可以复用相同模块在不同测试场景中，例如登录模块可用于多个功能测试。

模块化设计帮助测试框架更容易适应应用迭代，同时便于团队协作和管理。

### 2.6.2 Page Object 设计模式

Page Object 模式是一种将页面元素和操作方法封装在对象中的设计方法<sup>[19]</sup>。在手机 APP 自动化测试中，每个页面或功能模块可以对应一个 Page Object 类，类中包含：

- **元素定位方法：**定义按钮、输入框、列表等元素的定位方式。
- **操作方法：**封装用户可能执行的操作，如点击、输入、滑动等。

使用 Page Object 模式可以将页面结构与测试逻辑分离，提高测试脚本的可维护性和可读性。当应用界面发生变化时，只需修改 Page Object 中的元素定位或操作方法，而不必修改所有测试用例。

### 2.6.3 数据驱动测试

数据驱动测试是指将测试数据与测试逻辑分离，通过外部数据文件提供输入数据<sup>[20][21]</sup>。测试脚本根据数据文件自动执行不同的测试场景，而无需修改代码。优势包括：

- **覆盖更多场景：**同一测试脚本可以通过不同数据执行多种测试情况。
- **提高复用性：**测试逻辑与数据分离，使脚本可重复使用。
- **便于维护：**更新测试数据时无需改动测试脚本本身。

在手机 APP 自动化测试中，数据驱动测试尤其适合表单输入、登录验证、搜索功能等需要多种输入组合的功能测试。

### 2.6.4 日志记录与测试报告

在自动化测试过程中，日志记录和测试报告是验证测试结果和排查问题的重要手段。日志可以记录每一步操作、输入数据、执行时间、异常信息等，方便测试人员分析失败原因。测试报告则汇总测试结果，统计成功率、失败用例和错误信息，为团队提供直观的反馈。通过日志和报告，测试者能够快速发现问题，评估应用质量，并为持续集成提供可量化的数据支持

## 2.7 本章小结

本章先给大家介绍手机 APP 测试的基本概念和主要的测试类型，具体包含功能测试、性能与稳定性测试、兼容性测试、安全性测试和用户界面测试，同时，本章也说了自动化测试的优势和它的不足，也介绍了 Appium、Selenium 这两个工具，以及 Android Studio 模拟器和真实设备的测试环境。通过对这些内容的整理发现，在不同设备和不同环境里做手机 APP 测试的基础方法，还有要留心的问题，之后，本章还讲了自动化测试框架设计的一些相关理论，像模块化设计、



Page Object 模式、数据驱动测试，还有日志和报告记录这些内容。这些理论给后面具体的测试框架设计和实现给出了参考，能让测试脚本更容易维护、也能重复使用，还方便记录还有分析测试的结果。

## 第三章 手机 APP 自动化测试需求分析

### 3.1 测试对像

本文选了 ApiDemos6.0.10 当成手机 APP 自动化测试的对象。这个应用由 Appium。官方给出，是一个面向 Android 平台的示例应用，里面有丰富的界面元素和典型功能模块，比如按钮、文本输入框、列表、滑动视图、开关控件这些，很适合用来练习自动化测试。选 ApiDemos 的原因如下。

1. **功能完整：**覆盖常见的 Android 控件和操作流程，适合测试登录、表单填写、页面跳转、功能按钮点击这类核心功能。
2. **环境支持：**可以在 Android 模拟器和真实设备上运行，便于测试多设备、多分辨率以及不同系统版本下的适配性。
3. **易于自动化：**Appium 官方示例，社区支持强，能方便地编写测试脚本和进行结果验证。
4. **可复用性高：**模块化设计良好，测试场景可重复使用，适合验证自动化测试框架的可维护性和稳定性。

通过使用 ApiDemos 作为测试对象，可以在保证实验可行性的前提下，系统地设计和实现手机 APP 自动化测试的各类用例，为论文后续章节的框架搭建、脚本实现和测试结果分析提供可靠支撑。

### 3.2 测试环境

为了保证 ApiDemos 6.0.10 的自动化测试能够顺利进行，本文搭建了以下测试环境：

1. 操作系统与设备
  - **Android 模拟器：**使用 Android Studio 内置的模拟器（本文使用 Android 16.0），便于快速测试不同版本的 AndroidOS。

- **真实设备：**选用 Android 13.0 手机进行测试，以获取真实性能和硬件交互的结果，保证测试结果更接近用户实际使用环境。

## 2. 自动化工具

- **Appium：**作为核心自动化测试工具，用于控制模拟器和真机，实现点击、输入、滑动、页面跳转和元素校验等操作。
- **Selenium WebDriver：**作为辅助工具，提供 WebDriver 风格的接口，方便在移动 Web 或混合应用场景中调用自动化命令。

## 3. 开发与运行环境

- **编译语言：**Java
- **开发环境：**Eclipse
- **依赖库与工具：**Appium Java Client、ADB（Android Debug Bridge）、Appium Inspector

通过上述测试环境，可以在不同设备和操作系统版本下执行自动化测试，验证应用的核心功能、界面交互、性能表现和兼容性。同时，该环境为后续构建自动化测试框架、编写测试脚本和记录测试结果提供了坚实基础。

## 3.3 测试需求

本文在 ApiDemos 6.0.10 上进行自动化测试，测试需求主要包括功能测试、UI 测试、兼容性与性能测试以及随机稳定性测试。其中，功能测试和 UI 测试主要用于验证应用在固定操作流程下的功能正确性和界面交互效果；兼容性与性能测试主要用于观察应用在不同设备和系统版本下的运行情况；随机稳定性测试则通过 ADB Monkey 向应用发送大量随机事件，用于验证应用在非预期操作和随机交互场景下是否能够保持稳定运行。

### 3.3.1 功能测试需求

功能测试主要验证应用的核心功能是否按预期工作，包括：

- **登录和表单模块：**文本框输入、按钮点击、提交操作以及错误提示验证。

- 页表跳转模块：点击按钮或链接后能正确跳转到对应页面，页面元素显示正确。

### 3.3.2 UI 测试需求

UI 测试主要关注界面元素的显示与交互体验：

- 页面元素是否可见、可点击
- 布局、字体、颜色和图标位置是否符合预期
- 页面在不同分辨率或屏幕尺寸下的显示效果
- 滑动、滚动、动画效果等交互是否顺畅

### 3.3.3 兼容性与性能需求

兼容性和性能测试确保应用在多设备、多系统版本下运行稳定：

- 兼容性：验证应用在不同 Android 版本和屏幕分辨率上的稳定性
- 性能：页面加载速度、响应时间和整体稳定性
- 硬件依赖功能：例如传感器、摄像头等控件在真实设备上的表现
- 随机稳定性测试：通过 ADB Monkey 向应用发送随机点击、滑动和按键事件，观察应用是否出现崩溃、无响应或异常退出

## 3.4 测试用例设计

根据前节 3.3 中提出的测试需求，本文在 ApiDemos 6.0.10 上设计了典型自动化测试用例。测试用例涵盖登录、表单、页面跳转、功能按钮、UI 元素操作以及兼容性与性能模块，确保功能完整性、界面交互以及多设备兼容性。

表 3-1 功能测试模块

| 编号  | 测试模块 | 操作步骤                             | 预期结果                      |
|-----|------|----------------------------------|---------------------------|
| F01 | 登录模块 | 在 Text 模块输入文本，点击 Button          | 页面跳转到目标 Activity，页面元素显示正常 |
| F02 | 表单模块 | 在 EditText 输入文本，选择单选/复选控件，点击提交按钮 | 页面跳转或显示区域更新，输入和选择操作被正确执行  |
| F03 | 页面跳转 | 点击导航按钮或 ListItem                 | 页面跳转成功，目标页面元素可见           |

表 3-2 UI 测试模块

| 编号   | 测试模块   | 操作步骤           | 预期结果             |
|------|--------|----------------|------------------|
| UI01 | 元素可见性  | 打开页面，观察元素      | 所有控件和文字均可见       |
| UI02 | 布局测试   | 在不同分辨率模拟器上打开页面 | 页面布局、字体和控件位置保持一致 |
| UI03 | 滑动与滚动给 | 滑动列表、滚动页面      | 滑动顺畅，无异常，控件响应正确  |

表 3-3 兼容性与性能测试模块

| 编号  | 测试模块   | 操作步骤                   | 预期结果           |
|-----|--------|------------------------|----------------|
| C01 | 系统版本兼容 | 在不同 Android 模拟器上运行 APP | 页面正常启动，控件响应正常  |
| C02 | 性能测试   | 打开多个页面，执行操作            | 页面加载顺畅，控件响应及时  |
| C03 | 硬件功能   | 打开硬件功能页面, 操作控件         | 功能控件可操作，操作反馈正常 |

表 3-4 Monkey 随机稳定性测试模块

| 编号  | 测试模块           | 操作步骤                                    | 预期结果                     |
|-----|----------------|-----------------------------------------|--------------------------|
| S01 | Monkey 随机稳定性测试 | 使用 ADB Monkey 向 ApiDemos 发送随机点击、滑动和按键事件 | 应用正常运行，无崩溃、无 ANR、无异常退出   |
| S02 | 随机事件压力测试       | 设置较多随机事件数量，并加入事件间隔执行测试                  | 应用能够完成随机事件测试，日志中无严重错误    |
| S03 | 随机操作日志分析       | 保存 Monkey 执行日志，检查是否存在 Crash、ANR 或异常信息   | 能够根据日志判断应用稳定性，并为后续分析提供依据 |

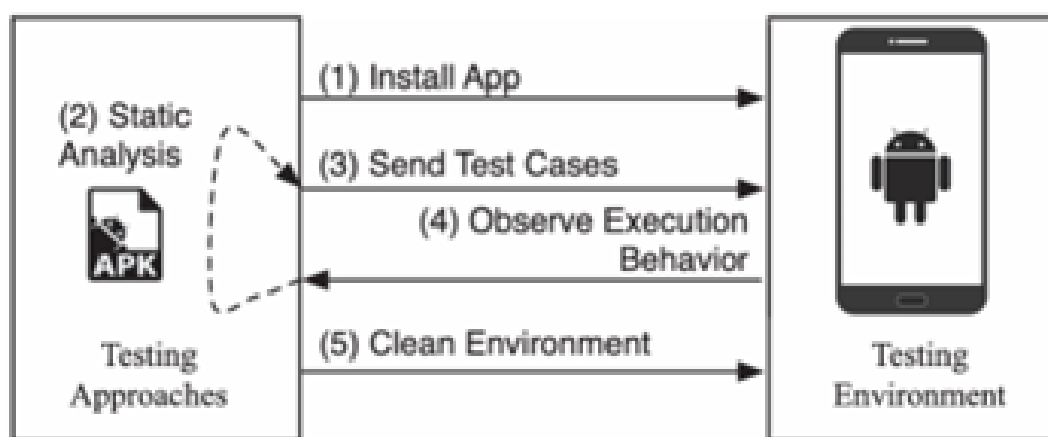


图 3-1 测试手机 APP 过程<sup>[1]</sup>

### 3.5 本章小结

本章围绕 ApiDemos 6.0.10，对手机 APP 自动化测试需求进行了分析。首先，从测试对象、测试环境以及测试条件出发，明确了移动应用在不同设备和系统版本下的运行特点和注意事项。随后，将测试需求划分为功能测试、UI 测试、兼容性与性能测试三个主要模块，并在此基础上补充了基于 ADB Monkey 的随机稳定性测试需求，进一步说明了各类测试的目标和重点关注点。

在此基础上，本文设计了典型的自动化测试用例，覆盖登录、表单、页面跳转及界面交互等模块。每条用例都明确了操作步骤和可验证的预期结果，为后续第四章自动化测试框架设计提供了可靠的输入依据。同时，ADB Monkey 随机稳

定性测试能够模拟用户非固定操作行为，用于观察应用在随机点击、滑动和按键等事件下是否出现崩溃、无响应或异常退出等问题。

通过本章的分析与设计，为自动化测试脚本的编写和执行奠定了基础，也为第五章测试实现与结果分析提供了更加完整的测试依据。同时，这些测试需求和用例设计也有助于提高测试结果的可靠性、可复用性和多设备适配能力。

## 第四章 手机 APP 自动化测试框架设计

本章基于第三章提出的测试需求和典型测试用例，结合第二章的自动化测试理论与工具技术，设计并搭建了适用于 ApiDemos 6.0.10 的移动应用自动化测试框架。框架以 Appium 为核心，辅以 Selenium WebDriver，并结合模块化设计思想、Page Object 模式、数据驱动测试以及日志与报告生成机制，旨在提高测试效率、脚本复用性及可维护性。在总体设计上，框架可以分为五个主要层次（如图4-1）：

1. 测试用例管理层：组织和管理测试用例，包括按功能模块或场景分类。支持参数化和数据驱动，使同一测试用例可在不同数据集上重复执行
2. 页面元素封装层：面或控件封装为独立类，提供点击、输入、滑动、跳转等操作。元素定位使用 Appium 的 id、Accessibility ID、XPath 等方式
3. 公共方法层：封装通用操作，如等待、截图、异常处理和日志记录。提高脚本复用性与可维护性
4. 配置管理层：管理测试环境参数，包括模拟器/真机选择、Android 版本、分辨率等。管理测试数据路径、参数文件和设备信息
5. 日志与报告层：自动记录测试日志并生成报告。支持错误截图和 HTML/Excel 输出，便于分析和持续测试

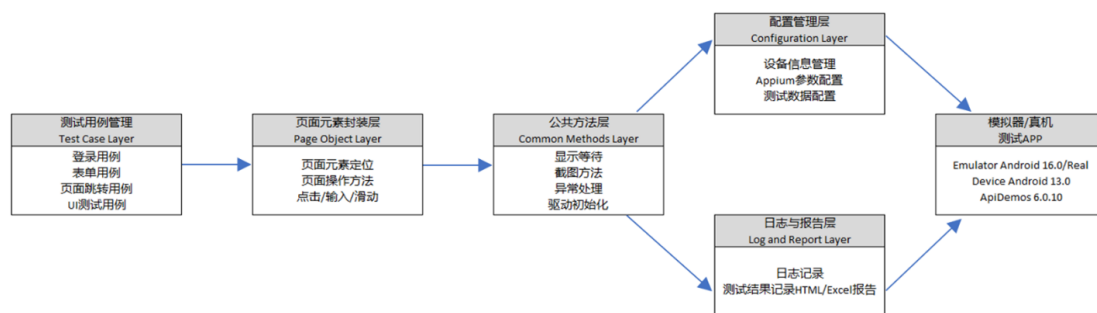


图 4-1 总体架构设计



## 4.1 测试用例管理层

测试用例管理层是移动应用自动化测试框架里的核心启动环节，主要作用是搞定测试用例的统筹编排、分类管控还有执行调度，本次研究用到的测试用例覆盖了登录、表单、页面跳转、UI 元素交互四个模块，这些模块包含应用的核心功能和关键交互逻辑，可以保证测试范围覆盖用户高频使用场景还有界面交互的主要路径。为提高运维管理的效率，所有测试用例都按照所属功能模块或者应用场景做好归类，其中登录相关用例统一归到 F01 组，表单操作用例归到 F02 组，页面跳转用例归到 F03 组，各个组里的用例还能结合功能属性或者测试目标做二次划分，既方便测试人员快速查找，也给后续测试范围的拓展和调整留出了空间。

测试用例管理层的架构设计优先同时考虑可复用特性还有适配的灵活性。框架自带参数化和数据驱动执行机制，测试数据可以从 JSON 这类外部文件导入，调整数据组合就能做到单条测试用例多轮反复执行，不用重复开发测试脚本就能覆盖更多输入场景，这一层级还负责测试流程的调度工作，能把执行指令发送到页面元素封装层和公共方法层来触发具体操作，保证每一条测试用例在不同硬件设备、不同操作系统版本环境里都能稳定运行，这种设计模式不光提高了整体测试执行的效率，还减少了没必要的重复开发投入，让框架能更好适配应用快速迭代更新的需求。

因此，测试用例管理层在整套自动化测试框架里处于核心位置，作用没法被替代，这个层级不光做到了测试用例规范化管理，也保证了测试的高覆盖度，还给后续页面元素封装、公共方法调用、日志和测试报告生成模块给出了标准化接口和清晰的调用逻辑。靠着这个层级的管控机制，所有测试脚本都能在框架里按照预设逻辑有序执行，同时也给测试结果的记录、分析还有持续测试流程搭好了可靠支撑，给整套自动化测试体系的高效运转还有框架长期可维护性提供了核心保障。

## 4.2 页面元素封装层（Page Object 模式）

页面元素封装层是框架的核心模块，利用 Page Object 模式完成应用界面控件和交互逻辑的封装工作，它最核心的设计思路，是把单个页面或者独立功能

模块抽象成专门的类，类里面整合了对应页面元素的定位逻辑和常用交互方法，包括按钮点击、文本输入、滑动屏幕、跳转界面、检查元素状态这类常见操作。用这种封装方法之后，测试用例不用直接调取界面元素，只需要调用对应页面对象的封装好的方法就能完成操作，既做到了测试脚本和界面元素的低耦合，也能大幅减少后续界面更新对现有脚本的影响。

本框架支持好几种页面元素定位方式，具体有 `id`、`Accessibility ID`、`XPath`、`class name`、`text` 这些类型，开发人员可以按照元素的属性特点，选出最合适的定位方法，比如常用控件可以优先用 `id` 定位，动态生成的元素就用 `XPath` 来定位，如果有跨平台测试的需求，就用 `Accessibility ID` 来完成适配。经过统一封装的定位逻辑，不仅能提高测试脚本读起来更清楚，也给不同测试用例重复使用同类操作流程带来了方便。

页面元素封装层还能和公共方法层联动调用，整合了等待机制、异常捕获、自动截图这些通用工具方法，用来保证操作运行的稳定。实际开始测试运行的时候，测试用例只要调用页面对象对外放出的接口发起操作请求，框架就会自动做完元素定位、动作执行和异常拦截的一整个流程处理，这种封装模式不光给自动化测试搭好了灵活又可靠的运行基础，也给之后做数据驱动测试、多设备适配测试提供了必要的支持。

## 4.3 公共方法层

公共方法层在自动化测试框架的中间位置，负责给测试用例管理层、页面元素封装层提供通用功能的方法和工具，它把框架里重复用到的操作逻辑打包起来，不用每次写重复代码，能提高测试脚本的复用性和可维护性。在这次研究的框架里，公共方法层主要提供控件等待、异常处理、截图、日志记录还有驱动初始化这类操作。

控件等待机制是公共方法层很重要的一个功能，它能保证测试脚本操作某个控件之前，先等这个控件在界面上完全加载完，这么做能避开界面渲染慢或者网络问题引发的操作失败，进而提高自动化测试的稳定程度，异常处理机制则用来抓测试过程里可能冒出来的各类错误，比如找不到元素、操作超时、应用崩溃这些情况。靠异常捕获，框架能给出对应的处理办法，比如再试一次、记下来错

误信息或者跳过这一步，让测试过程不会中断，同时也能给排查问题提供依据。

另外，公共方法层还给出了截图和日志记录功能，能把测试执行的过程详细记下来。每次操作或者异常出现的时候，框架会自动截屏幕图、写到日志文件里，方便测试人员之后做分析和调试，同时，驱动初始化方法也放在公共方法层里实现，用来统一管 Appium 或者 Selenium WebDriver 的启动、配置和关闭流程，这样各个测试用例不用重复配置驱动参数，就能直接跑自动化操作。

借助公共方法层，整个自动化测试框架做到了低耦合、高内聚的设计思路，测试用例不用管每个基础操作具体怎么实现，只要调用公共方法就能做完复杂动作，这么做不光提高了测试脚本的开发效率，也保证了框架在功能扩展和维护的时候好操作。公共方法层是框架的基础支撑，给后面的配置管理层和日志报告层给出了可靠的操作接口，同时也给多设备、多版本和多场景的自动化测试执行提供了稳定保障。

## 4.4 配置管理层

配置管理层是自动化测试框架里衔接上下层的核心模块，主要负责测试全流程里各类配置参数和环境信息的统一管理工作，它要管的内容包括测试设备参数、操作系统版本号、应用安装路径、测试数据存储位置、Appium 和 Selenium 这类驱动的运行参数等等。把这些配置集中管控之后，测试用例和页面元素封装层就能在不同硬件设备、系统版本还有测试场景下稳定运行，不用测试人员每次跑测试之前手动改参数或者动底层代码。

本框架的配置管理层用外部配置文件来存各类参数数据。测试用例启动运行之前，会自动读取这个配置文件的内容，还能根据不同的测试需求自动匹配模拟器或者实体测试设备、对应屏幕分辨率、操作系统版本以及测试数据调用路径，这种架构不光提高了测试环境切换的灵活度，还给团队在多台设备、不同系统环境下进行回归测试和兼容性测试提供了方便。

配置管理层还拥有关键参数的动态调整和扩展能力。比如要新增测试终端或者迭代应用版本的时候，只需要改配置文件里对应的条目就行，不用调整现有的测试脚本，能大幅压缩框架日常维护要花的成本，框架靠配置管理层做到了测试环境和测试脚本的逻辑解耦，不光提高了测试执行过程的标准化程度还有可

控性，也给日志记录层运行以及测试报告生成提供了稳定的基础数据来源。

整体来看，配置管理层给自动化测试框架在多环境、多设备、多版本场景下的运行稳定性还有资源复用性提供了有力保障，是支撑整套框架正常运转的核心模块之一。这个模块不光简化了测试环境的前期配置流程，还给后续测试结果分析和持续性测试落地提供了可靠支撑，对框架的功能扩展和长期运维都有很重要的基础作用。

## 4.5 日志与报告模块

日志与报告模块属于自动化测试框架里很关键的一块，主要要记下测试过程里的操作信息、出问题的情况还有测试结果，最后还要生成方便分析报告，这个模块不光能帮测试人员追踪找出问题，还能对测试结果做汇总统计，给测试反馈还有后续优化打好基础。在整个框架里边，日志与报告模块和测试用例管理层、页面元素封装层还有公共方法层配合得很紧密，保证每一次测试运行都能被准确记下来、分析好。

具体做实现的时候，框架会在测试用例执行的每一步都写下日志，内容包括开始执行的时间、正在执行的测试步骤、操作的对象还有最终执行结果。碰到异常情况，比如找不到控件、操作失败又或者应用直接崩溃了，框架会自动抓下来记下详细信息，同时生成截图存到指定的文件夹里，这些日志跟截图能帮测试人员更快找到问题，减少花在排查上的时间，提高调试的效率，同时，框架会把测试结果汇总起来生成可视化报告，像 CSV 或者 Excel 格式，报告里会写清楚测试用例状态、执行用了多久、有多少错误还有截图的存放链接，方便整个团队查看分享。

另外，日志与报告模块还可以和持续集成（CI/CD）系统搭配使用，做到自动化测试结果持续记录和分析，测试脚本在新的构建版本上运行的时候，框架能自动更新报告，把不同版本之间测试结果的变化趋势展示出来，给回归测试还有版本评估做参考。利用这个模块，测试结果不光能被准确记下来，还能长期保存，用来做统计分析还有质量评估，给团队制定测试策略还有优化方案给出数据支持。

## 4.6 本章小结

本章围绕移动端 APP 自动化测试框架的整体设计思路展开讲解，逐一对每个功能模块的核心定位还有实现逻辑做了拆解，测试用例管理模块负责测试用例的统筹调度，能够适配多种业务模块、多种测试场景的执行要求，利用参数化配置跟数据驱动机制，大幅提高了用例的复用效率，也让迭代维护变得更方便。页面元素封装模块按照 **Page Object** 设计模式，把界面元素属性和对应的交互操作封装成独立对象，最大程度减少测试脚本跟界面迭代之间的耦合关联，有效增强框架的运行稳定性和功能可扩展性。公共方法模块把等待机制、异常捕获、操作截图、驱动初始化这类通用能力整合在一起，给全流程测试执行提供了底层功能支持，配置管理模块把测试设备、适配系统版本、运行参数这些内容做统一管理，保证测试任务能在多终端、多系统版本的不一样环境里稳定运行，日志与报告模块会全程记录测试执行的完整过程和最终结果，自动生成可视化测试报告，给问题定位排查、进行回归测试还有迭代优化测试方案提供数据支持。

结合上述各个模块的配合设计，这套自动化测试框架拥有了分层解耦、模块化拆分、高复用性的架构特点，从底层给测试执行全流程的稳定性和可维护性做了保障。这个框架除了能完成常规功能逻辑、界面交互场景的自动化校验之外，还给后续扩展多终端适配、落地数据驱动测试还有对接持续集成流程留出了实现空间，整体来看，本章提出的设计方案给第五章测试落地实现还有结果分析环节做足了前期准备，也为完整呈现自动化测试效果搭好了核心框架的支撑。

## 第五章 手机 APP 自动化测试实现与结果分析

本章内容围绕实际手机 APP 测试中的实现过程及测试结果分析。内容包含三个部分，分别为测试环境搭建、自动化脚本的编写与执行、典型测试用例的运行结果以及对测试效果的分析。通过本章的描述，能直观了解自动化测试框架在各类功能模块、界面交互以及兼容性测试中的实际应用情况，并为下一章总结框架的有效性和改进方向提供依据。

首先，将介绍实验所使用的测试环境，包括硬件设备、软件工具及应用版本等信息，为测试的可复现性提供基础。随后，详细说明如何将第三章设计的测试用例在框架中实现为可执行的自动化脚本，并结合页面元素封装层和公共方法层进行操作。接着，通过对典型功能和界面测试用例的执行结果进行整理与展示，评估框架在功能验证、界面交互、兼容性和稳定性方面的表现。最后，对测试结果进行分析，总结自动化测试在实际应用中的优势、存在的问题以及改进建议，为后续总结与展望提供参考。

### 5.1 测试环境搭建

本次研究整套测试环境都是在 Windows 11 操作系统上搭建完成，一共包括开发运行环境、测试工具、移动终端配置。开发端使用 Eclipse IDE 作为核心开发工具，配合 Java 21 版本来做项目的编译和运行工作，整个项目依靠 Maven 对所有依赖做统一管理，保证 Appium、Selenium 这类自动化测试类库可以顺利整合进去。测试终端同时用了 Android 模拟器和真机，既可以快速验证功能，也能还原真实使用场景的测试需求。具体的环境参数配置整理如下：

1. 开发工具：Eclipse IDE (2025-12 版本)，支持 Java 21 编译，便于管理项目结构和依赖库。
2. Java 环境：JRE 及 JDK 版本均为 21，保证与 Appium Java 客户端的兼容性。
3. 自动化测试工具：Appium 3.2.1 作为核心自动化框架，同时有 Selenium Web-

Driver 进行辅助操作。

4. 移动设备环境：使用 Android Studio 模拟器（Android 16.0）和真机（Android 13.0）设备进行测试。模拟器方便快速验证功能，真实设备用于性能和兼容性验证。
5. 项目管理：通过 Maven 管理依赖库，保证测试脚本可复用性和跨平台兼容性。

## 5.2 自动化测试脚本实现

测试环境布置好之后，本次研究选了 ApiDemos 6.0.10 当作载体，完成了典型自动化测试脚本的开发，用来校验核心功能模块，这个自动化脚本用 Appium 和 Selenium WebDriver 做开发框架，选了 Java 当开发语言，基于 Eclipse IDE 做完调试和运行环节，脚本用了模块化的设计思路，把每个功能模块都封装成独立方法，能提高复用性还有可维护性，具体要做的内容有这些。

- 登录功能验证：自动做完文本输入、按钮点击操作，检查目标 Activity 页面跳转有没有成功，同时确认界面元素的显示状态是不是正常。
- 表单输入和操作：往输入框里填入测试数据，提交之后检查页面返回的结果，确认整个表单操作流程的逻辑没有错。
- 页面跳转：模拟用户在多个页面之间切换导航的操作，检查跳转逻辑跟预设规则是不是能对上。
- UI 元素操作：做完按钮点击、页面滑动、选项卡切换等操作，验证界面元素可以正常响应各类触发动作。
- 结果记录：调用日志记录和截图工具，保存脚本运行状态和输出结果，给之后分析问题、做回归测试给出数据支撑。

自动化脚本运行的时候，可以反复核对核心功能模块的逻辑，能适配多设备、多分辨率的测试场景。基于这个脚本，测试团队能在比较短的时间里完成多个功能点的覆盖测试，也能保证测试输出结果稳定，之后也能重复用，给下一章节的测试结果分析工作打好基础。

开发自动化测试脚本的时候，先采集应用程序界面元素的定位信息，做元素分析的时候，可以借助 Appium Inspector 工具处理 ApiDemos 应用，先把模拟器

或者物理真机的链路对接好，再提取当前页面里各个控件的属性参数，这些参数包含 id、XPath、Accessibility ID、class name 等多项内容。测试人员可以用这个工具直接看到界面的层级结构，再结合实际场景挑合适的定位方法，完成后面的元素交互操作。

实际测试的时候，id 定位方法的稳定性和运行效率表现都比较好，一般都会被当成元素定位的首选方案；要是碰到界面结构复杂、元素属性会动态变化的场景，就可以搭配 XPath 和 Accessibility ID 当补充的定位方法，用 Appium Inspector 采集到的元素属性信息可以直接导入自动化脚本里使用，能支持完成按钮触发、文字录入、页面切换、界面元素合法性校验等各种操作。对于部分动态或复杂界面元素，本文结合使用 XPath 进行辅助定位，以保证自动化操作的准确性和稳定性。

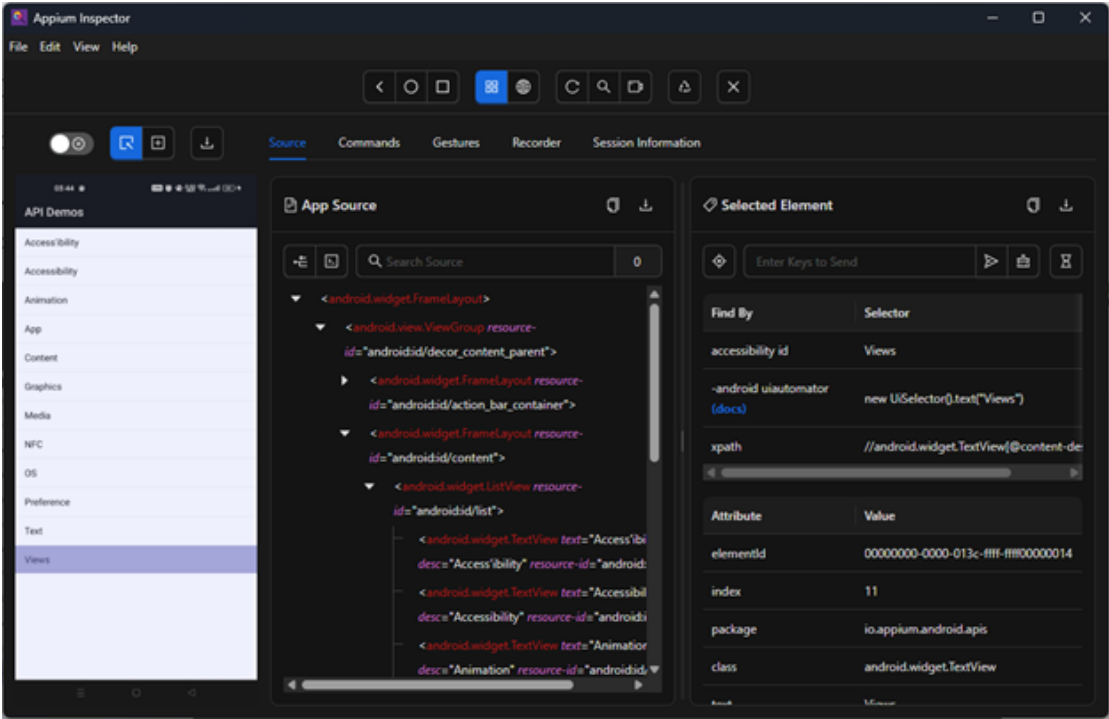


图 5-1 Appium Inspector 获取元素信息示意图

### 5.3 典型测试用例执行

选取 ApiDemos 6.0.10 的登录模拟功能作为典型测试用例执行，目的是验证自动化框架在核心功能和界面交互方面的实际效果。测试脚本基于 Appium + UiAutomator2 编写，运行于模拟器与真机并通过 AndroidDriver 控制应用操作。



测试分别在 Android 13 真实设备（OPPO CPH2025）与 Android 16 模拟器两种环境下运行，以评估框架的跨设备兼容性。

### 5.3.1 登陆测试

登录功能是移动应用里最基础也最关键的用户交互入口，因此本文将登录模拟测试作为首个典型用例进行验证。本次测试选了 ApiDemos 6.0.10 的 Views→TextFields 功能模块当测试对象，通过自动化脚本模拟用户输入用户名、密码还有附加信息的操作，来检验对应框架在文本输入、页面跳转、控件定位方面的执行能力。图5-2展示了本测试用例的脚本整体结构。



图 5-2 登录测试脚本结构

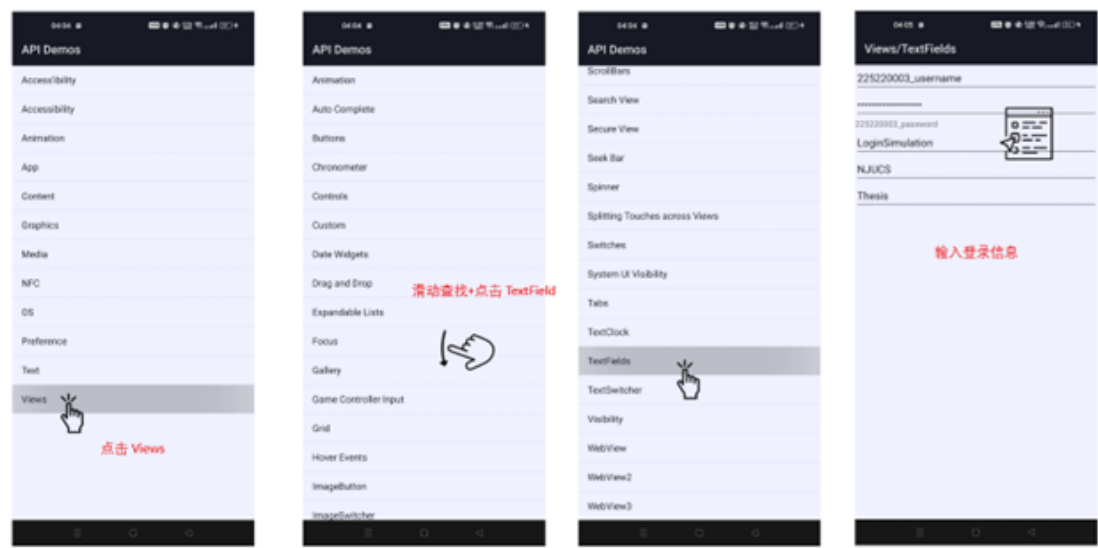


图 5-3 真机 Android 13 执行登录测试用例

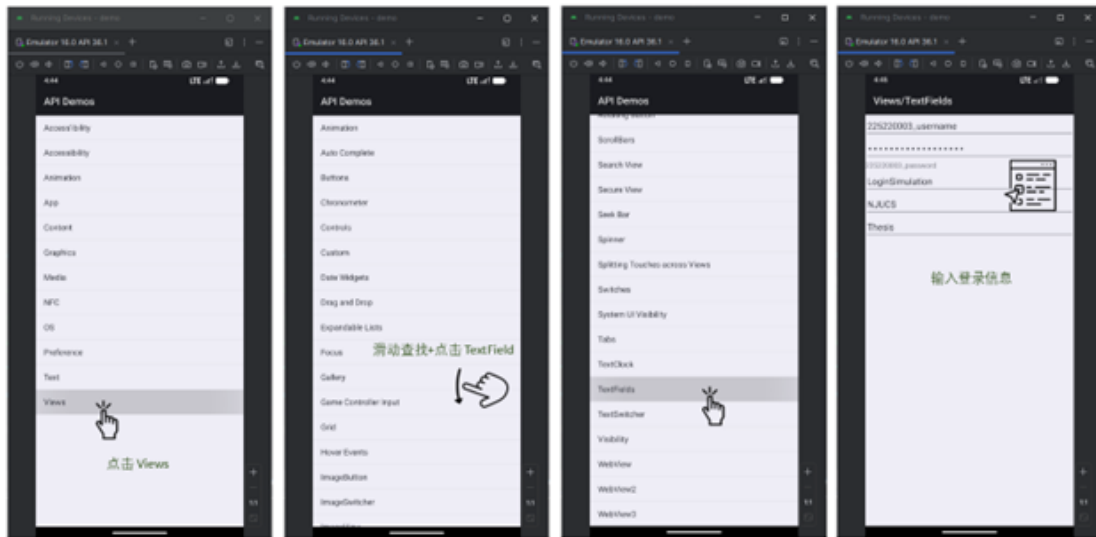


图 5-4 拟器 Android 16 执行登录测试用例

以下代码展示了登录模拟测试用例的完整实现，包含驱动初始化、设备能力配置及界面元素操作三个核心部分。

---

**Algorithm 1** Pseudocode for Login Function Testing

---

**Data:** Mobile application ApiDemos 6.0.10, real device/emulator

---

**Result:** Automated login test executed successfully

---

- 1 Initialize desired capabilities: device name, platform, version, automation engine, app package and activity Connect to Appium server at localhost:4723 Launch the application using AndroidDriver
- 2 Locate "Views" element and click it Scroll to "TextFields" element and click it
- 3 Fill input fields:
  - Username: "225220003\_username"
  - Password: "225220003\_password"
  - LoginSimulation: "LoginSimulation"
  - School: "NJUCS"
  - Thesis: "Thesis"

Print "Pass" to console. Quit the driver

---

上述代码可分为三个核心部分。第一部分为驱动初始化与设备能力配置，通过 UiAutomator2Options 实例来设置目标设备标识、运行平台类型、Android 系

统版本、自动化测试引擎、被测应用包名还有启动入口 Activity 的参数，保证 Appium 服务能准确识别测试设备，还能和设备建好有效连接。第二部分负责连接 Appium Server 并启动 AndroidDriver，脚本通过本地 4723 端口和 Appium 服务端完成数据交换，之后设置了 2 秒的等待时间，等被测应用完成启动加载。第三部分为界面元素操作，脚本按顺序走完“Views”菜单定位、跳转到“TextFields”输入页面这些操作，再给页面里的五个输入框依次填入用户名、密码及附加信息，最终输出” Pass”并调用 driver.quit() 退出驱动，完成整个登录模拟流程。执行结果：

- 所有步骤均在真实设备上成功执行，应用界面响应正常，无崩溃或卡顿。
- 文本输入功能正常，每个输入框都能正确显示输入内容。
- 滑动和点击操作顺利，界面跳转准确到目标模块。

通过该用例，可以验证自动化测试框架在核心功能操作、页面跳转及输入验证方面的有效性，为后续其他模块的测试和框架评估提供了可靠依据。

### 5.3.2 表单以及元素可见性测试

表单控件交互与界面元素可见性校验是移动应用测试的重要组成部分。本次测试选了 ApiDemos 6.0.10 的 Views→Radio Group 模块当作测试对象，设计并运行了 F02（表单元素交互）与 UI01（元素可见性与动态内容校验）两类测试用例。测试目标为：通过自动化脚本依次点击 Clear 按钮及 Snack、Breakfast、Lunch、Dinner 四个单选按钮，每做完一次操作就同步读取“ You have selected”文本标签的显示内容，验证界面元素可以对用户操作给出正确响应，而且动态提示文本能跟着选项切换做到实时更新，脚本用 sum 计数器统计执行通过的用例数量，完成多个子场景的自动化批量校验，本次测试用例的脚本整体架构可以看图 5-3。



图 5-5 表单以及元素可见性测试脚本结构

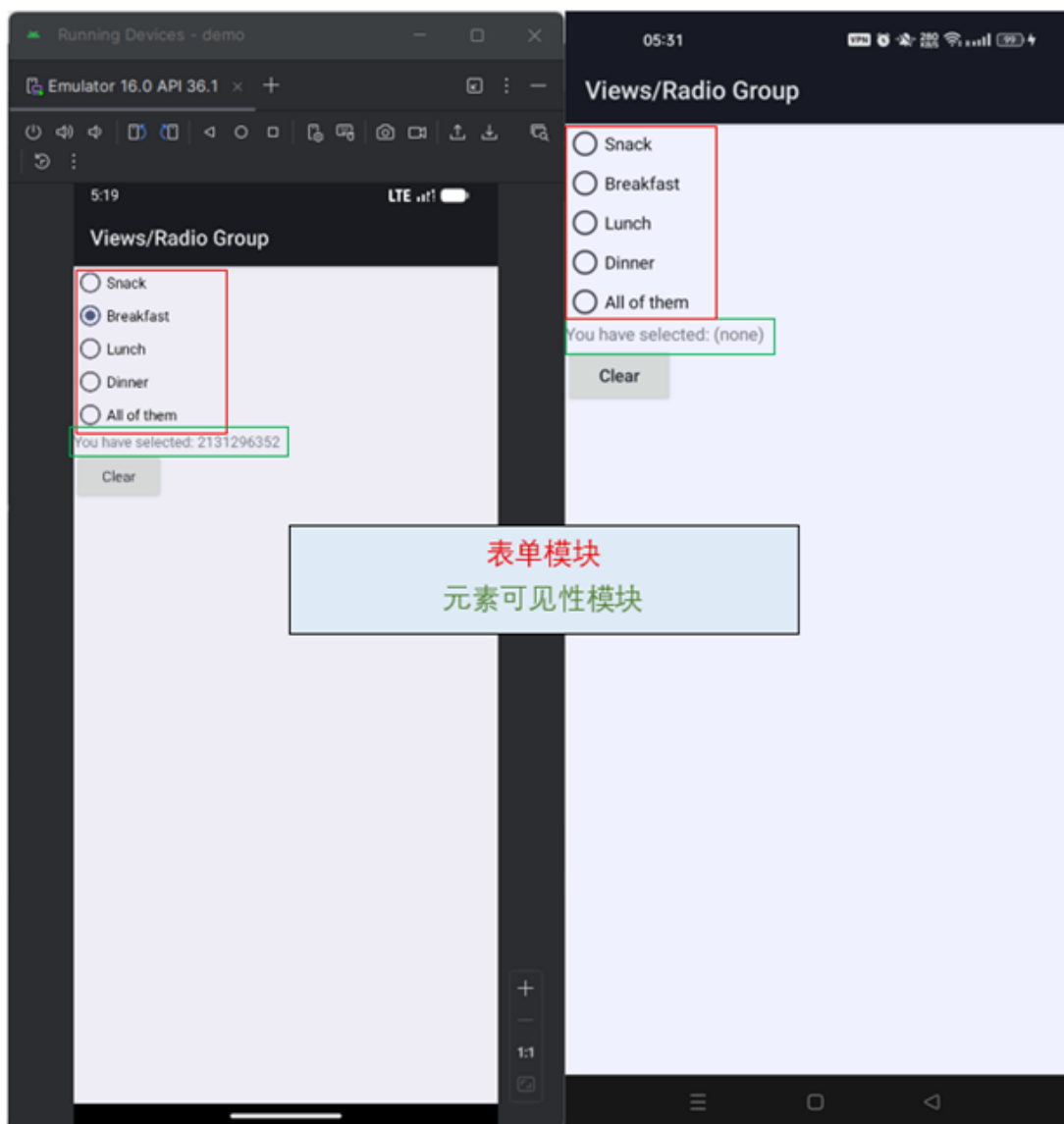


图 5-6 模拟器（左）和真机（右）执行表单以及元素可见性测试用例

---

**Algorithm 2** Pseudocode for Form and UI Visibility Test

---

**Data:** Mobile application ApiDemos 6.0.10, real device/emulator

**Result:** Automated RadioGroup button selection test executed successfully

- 1 Initialize desired capabilities: device name, platform, version, automation engine, app package and activity Connect to Appium server at localhost:4723 Launch the application using AndroidDriver
- 2 Navigate to "Views" page and click Scroll to "Radio Group" element and click
- 3 Test "Clear" button:
  - Click "Clear"
  - Verify that the result text shows "(none)"

Test radio button selections:

- Click "Snack", verify result text
- Click "Breakfast", verify result text
- Click "Lunch", verify result text
- Click "Dinner", verify result text

Count the number of passed test cases If all tests passed, print "All test cases: Pass", otherwise print how many passed. Quit the driver

---

上述代码初始化计数器 sum 用于统计断言通过数量。核心交互与断言逻辑：脚本首先通过 UiScrollable 滚动定位 Radio Group 入口并进入，随后依次点击 Clear 按钮及四个单选按钮（Snack、Breakfast、Lunch、Dinner），每次操作后通过 XPath 定位 "You have selected" 文本元素并读取其内容与预期值进行比对。其中 Clear、Snack 与 Breakfast 三个子用例的断言已完整实现并通过验证；Lunch 与 Dinner 的操作步骤均正常执行，但断言逻辑在当前阶段处于调试状态，getText() 调用已临时屏蔽，待确认正确预期值后将完善补充。执行结果：

- Clear 按钮点击后，"You have selected: (none)" 文本正确显示，断言通过。
- Snack 与 Breakfast 单选按钮点击后，反馈文本内容均与预期 resource-id 一致，断言通过。
- Lunch 与 Dinner 单选按钮的点击操作顺利执行，界面响应正常，无崩溃或

卡顿；断言部分因调试需要暂时屏蔽，待预期值确认后将完整补全。

- 脚本通过计数器 `sum` 对已验证的子用例进行统计，当前阶段两个完整断言均通过，结果输出正常。

通过该用例，可以验证自动化测试框架在表单元素交互与 UI 元素可见性方面的执行能力，同时也体现了框架对多子场景批量验证与结果统计的支持。

### 5.3.3 结合综合测试

做完登录模拟测试（F01）、表单及元素可见性测试（F02、UI01）之后，本文设计并搭好了综合功能测试模块 `ActivityTest`，用来检查自动化测试框架在多个任务串起来执行的场景里，运行稳不稳、能不能覆盖到需要测试的场景。跟之前那两个单功能验证不一样，`ActivityTest` 把第三章提出来的三类测试需求都放到同一个测试套件里：功能测试这块，覆盖了自定义标题调整、页面跳转检查、跨 `Activity` 值回传、系统弹窗（`Alert Dialogs`）交互这些核心测试内容；UI 测试这块，靠跳转之后页面对内容做断言，来判断界面元素能不能看见、文字显示对不对，满足元素可见性和布局检查的测试要求，兼容性测试这块，把测试脚本分别放到 `Android 13` 实体设备和 `Android 16` 模拟器里跑，验证框架在不同系统版本之间能不能适配，稳不稳，靠连续的页面跳转和多层级的检查逻辑，`ActivityTest` 同时覆盖了功能、界面、兼容性这三类测试维度，能给全方位评估框架实际运行性能给出更充足的支撑依据。

与面两部分的测试脚本实现逻辑比，这部分测试脚本用了更成熟的模块化封装架构，也是本文里自动化测试框架的公共方法层、日志报告模块设计思路的具体呈现。所有测试操作步骤都靠统一的 `runStep` 方法管控，这个方法能接收模块标识、步骤名称、执行说明和具体操作逻辑作为输入参数，除了能做完操作执行、捕捉运行异常之外，还能自动把步骤标识、执行状态、异常信息和时间戳存到 `CSV` 格式的日志文件里，断言校验逻辑用 `checkEquals`、`checkContains`、`checkContainsAll` 三个工具方法落实，分别对应精确匹配、单个关键词检索、多个关键词同时匹配这三类检查场景，可以覆盖不同场景里对界面返回结果的检查需求。这个封装模式做到了操作逻辑和断言校验的解耦，同时把原来手动用 `System.out.println` 输出日志改成了结构化自动写入，既减少了人工汇总测试结果

带来的偏差，也给后面 5.4 节做测试结果统计分析给出了更完整、可信的原始数据支撑。

下面的伪代码展示了 ActivityTest 的核心实现逻辑，包含子模块导航、文本录入、断言校验和页面回退这些核心操作流程。

---

**Algorithm 3** Pseudocode for the Comprehensive Activity Test Assessment Procedure

---

**Data:** Mobile application ApiDemos 6.0.10, real device/emulator

**Result:** Overall test executed successfully

- 1 Initialize desired capabilities: device name, platform, version, automation engine, app package and activity Connect to Appium server at localhost:4723 Launch the application using AndroidDriver
  - 2 Execute Algorithm 1: Login test Execute Algorithm 2: Form input and element visibility test
  - 3 Start ActivityTest and record the start time
  - 4 Locate the "Activity" element and click it
  - 5 Locate the "Custom Title" element and click it. Input "NANJING" into the left title field. Input "UNIVERSITY" into the right title field. If the title text is correct, record the both title test as passed
  - 6 Locate the "Hello World" element and click it. Get the displayed text and compare it with "Hello, World!". If "Hello, World!" is displayed, record the test as passed.
  - 7 Locate the "Receive Result" element and click it. Select the "Corky" button. Select the "Violet" button. If both keywords are contained, record the both result test as passed.
  - 8 Locate the "Alert Dialogs" element and click it. Click the "OK Cancel dialog with a long message" button. Check whether the message contains the expected keywords "Plloaso" and "Uhex" If the expected keywords are contained, record the long message dialog test as passed;
  - 9 Record the total execution time, error point, error content and success rate in the CSV log file
  - 10 Print "ActivityTest Finished" Quit the driver
-

上面这段代码展示了 **ActivityTest** 模块下四个核心子操作的实现逻辑，其中 **Custom Title** 子模块会先清空输入框里原来的内容，填上要修改的目标标题之后触发修改按钮的点击事件，之后再把界面上显示的标题内容读取，和提前设置好的数值做精准匹配，验证界面文本修改功能的正确性。**Receive Result** 子模块模拟了跨 **Activity** 的数据传递场景，脚本通过多次触发结果选择操作，并验证结果文本是否包含预期的累积内容，体现了框架对跨页面状态传递与多步骤结果验证的支持能力。**Alert Dialogs** 子模块靠准确定位弹窗内部的长文本控件，对控件里的关键信息做检查，验证系统级弹窗内容的可读性与元素可访问性。所有子步骤的执行结果都由 **runStep** 组件自动整理好写到日志文件里，无需人工逐步记录，有效提升了多步骤测试场景下的结果管理效率。执行结果：

- 将 **LoginTest**、**RadioGroupTest** 与 **ActivityTest** 三个测试模块在同一 **main()** 方法中顺序执行，构成本文的综合测试集。整体测试在 **Android16** 模拟器与 **Android13** 真机两种环境下分别运行，所有步骤的执行状态、耗时与报错信息均通过 **runStep** 机制自动汇总至 **CSV** 日志文件，测试人员可直接查阅日志获取完整的执行记录，无需依赖控制台输出。图5-7和图5-8展示了综合测试全部通过时的日志执行结果与耗时统计。

| TestTime | TestName       | StepName                    | Status   | Duration(ms) | ErrorPoint | ErrorContent                      | PassedCases | TotalCases | SuccessRate |
|----------|----------------|-----------------------------|----------|--------------|------------|-----------------------------------|-------------|------------|-------------|
| #####    | LoginTest      | Click Views                 | PASS     | 1413         |            |                                   | 1           | 1          | 100         |
| #####    | LoginTest      | Click TextFields            | PASS     | 5851         |            |                                   | 2           | 2          | 100         |
| #####    | LoginTest      | Input TextFields            | PASS     | 3501         |            |                                   | 3           | 3          | 100         |
| #####    | RadioGroupTest | Back To Main Page           | PASS     | 2331         |            |                                   | 4           | 4          | 100         |
| #####    | RadioGroupTest | Click Views                 | PASS     | 1775         |            |                                   | 5           | 5          | 100         |
| #####    | RadioGroupTest | Click Radio Group           | PASS     | 6634         |            |                                   | 6           | 6          | 100         |
| #####    | RadioGroupTest | Clear Button                | PASS     | 3403         |            |                                   | 7           | 7          | 100         |
| #####    | RadioGroupTest | Snack Button                | PASS     | 4703         |            |                                   | 8           | 8          | 100         |
| #####    | RadioGroupTest | Breakfast Button            | PASS     | 3956         |            |                                   | 9           | 9          | 100         |
| #####    | RadioGroupTest | Lunch Button                | PASS     | 831          |            |                                   | 10          | 10         | 100         |
| #####    | RadioGroupTest | Dinner Button               | PASS     | 1416         |            |                                   | 11          | 11         | 100         |
| #####    | RadioGroupTest | Back To Main Page           | PASS     | 3042         |            |                                   | 12          | 12         | 100         |
| #####    | ActivityTest   | Click App                   | PASS     | 2712         |            |                                   | 13          | 13         | 100         |
| #####    | ActivityTest   | Click Activity              | PASS     | 3425         |            |                                   | 14          | 14         | 100         |
| #####    | ActivityTest   | Click Custom Title          | PASS     | 4615         |            |                                   | 15          | 15         | 100         |
| #####    | ActivityTest   | Change Left Title           | PASS     | 8040         |            |                                   | 16          | 16         | 100         |
| #####    | ActivityTest   | Change Right Title          | PASS     | 4805         |            |                                   | 17          | 17         | 100         |
| #####    | ActivityTest   | Back From Custom Title      | PASS     | 673          |            |                                   | 18          | 18         | 100         |
| #####    | ActivityTest   | Hello World                 | PASS     | 1824         |            |                                   | 19          | 19         | 100         |
| #####    | ActivityTest   | Back From Hello World       | PASS     | 639          |            |                                   | 20          | 20         | 100         |
| #####    | ActivityTest   | Click Receive Result        | PASS     | 7559         |            |                                   | 21          | 21         | 100         |
| #####    | ActivityTest   | Click Get Result For Corky  | PASS     | 742          |            |                                   | 22          | 22         | 100         |
| #####    | ActivityTest   | Corky Button                | PASS     | 7882         |            |                                   | 23          | 23         | 100         |
| #####    | ActivityTest   | Click Get Result For Violet | PASS     | 180          |            |                                   | 24          | 24         | 100         |
| #####    | ActivityTest   | Violet Button               | PASS     | 8040         |            |                                   | 25          | 25         | 100         |
| #####    | ActivityTest   | Back To Activity Page       | PASS     | 3902         |            |                                   | 26          | 26         | 100         |
| #####    | ActivityTest   | Click Alert Dialogs         | PASS     | 1727         |            |                                   | 27          | 27         | 100         |
| #####    | ActivityTest   | Click Long Message          | PASS     | 154          |            |                                   | 28          | 28         | 100         |
| #####    | ActivityTest   | Long Message Check          | PASS     | 2056         |            |                                   | 29          | 29         | 100         |
| #####    | ActivityTest   | Click OK                    | PASS     | 152          |            |                                   | 30          | 30         | 100         |
| #####    | Summary        | All Test Cases              | FINISHED | 112690       |            | Total Passed: 30, Total Cases: 30 | 30          | 30         | 100         |

图 5-7 综合测试全部通过时的 **test\_log(3)** 执行结果（真机）



| TestTime | TestName       | StepName                    | Status   | Duration(ms) | ErrorPoint                        | ErrorContent | PassedCases | TotalCases | SuccessRate |
|----------|----------------|-----------------------------|----------|--------------|-----------------------------------|--------------|-------------|------------|-------------|
| #####    | LoginTest      | Click Views                 | PASS     | 1758         |                                   |              | 1           | 1          | 100         |
| #####    | LoginTest      | Click TextFields            | PASS     | 5962         |                                   |              | 2           | 2          | 100         |
| #####    | LoginTest      | Input TextFields            | PASS     | 2759         |                                   |              | 3           | 3          | 100         |
| #####    | RadioGroupTest | Back To Main Page           | PASS     | 1266         |                                   |              | 4           | 4          | 100         |
| #####    | RadioGroupTest | Click Views                 | PASS     | 1819         |                                   |              | 5           | 5          | 100         |
| #####    | RadioGroupTest | Click Radio Group           | PASS     | 4828         |                                   |              | 6           | 6          | 100         |
| #####    | RadioGroupTest | Clear Button                | PASS     | 725          |                                   |              | 7           | 7          | 100         |
| #####    | RadioGroupTest | Snack Button                | PASS     | 750          |                                   |              | 8           | 8          | 100         |
| #####    | RadioGroupTest | Breakfast Button            | PASS     | 727          |                                   |              | 9           | 9          | 100         |
| #####    | RadioGroupTest | Lunch Button                | PASS     | 756          |                                   |              | 10          | 10         | 100         |
| #####    | RadioGroupTest | Dinner Button               | PASS     | 730          |                                   |              | 11          | 11         | 100         |
| #####    | RadioGroupTest | Back To Main Page           | PASS     | 1744         |                                   |              | 12          | 12         | 100         |
| #####    | ActivityTest   | Click App                   | PASS     | 1811         |                                   |              | 13          | 13         | 100         |
| #####    | ActivityTest   | Click Activity              | PASS     | 1340         |                                   |              | 14          | 14         | 100         |
| #####    | ActivityTest   | Click Custom Title          | PASS     | 1333         |                                   |              | 15          | 15         | 100         |
| #####    | ActivityTest   | Change Left Title           | PASS     | 1778         |                                   |              | 16          | 16         | 100         |
| #####    | ActivityTest   | Change Right Title          | PASS     | 1742         |                                   |              | 17          | 17         | 100         |
| #####    | ActivityTest   | Back From Custom Title      | PASS     | 624          |                                   |              | 18          | 18         | 100         |
| #####    | ActivityTest   | Hello World                 | PASS     | 1813         |                                   |              | 19          | 19         | 100         |
| #####    | ActivityTest   | Back From Hello World       | PASS     | 636          |                                   |              | 20          | 20         | 100         |
| #####    | ActivityTest   | Click Receive Result        | PASS     | 1786         |                                   |              | 21          | 21         | 100         |
| #####    | ActivityTest   | Click Get Result For Corky  | PASS     | 1118         |                                   |              | 22          | 22         | 100         |
| #####    | ActivityTest   | Corky Button                | PASS     | 1754         |                                   |              | 23          | 23         | 100         |
| #####    | ActivityTest   | Click Get Result For Violet | PASS     | 2265         |                                   |              | 24          | 24         | 100         |
| #####    | ActivityTest   | Violet Button               | PASS     | 1184         |                                   |              | 25          | 25         | 100         |
| #####    | ActivityTest   | Back To Activity Page       | PASS     | 1177         |                                   |              | 26          | 26         | 100         |
| #####    | ActivityTest   | Click Alert Dialogs         | PASS     | 2349         |                                   |              | 27          | 27         | 100         |
| #####    | ActivityTest   | Click Long Message          | PASS     | 107          |                                   |              | 28          | 28         | 100         |
| #####    | ActivityTest   | Long Message Check          | PASS     | 915          |                                   |              | 29          | 29         | 100         |
| #####    | ActivityTest   | Click OK                    | PASS     | 626          |                                   |              | 30          | 30         | 100         |
| #####    | Summary        | All Test Cases              | FINISHED | 64468        | Total Passed: 30, Total Cases: 30 |              | 30          | 30         | 100         |

图 5-8 综合测试全部通过时的 test\_log(4) 执行结果（模拟器）

- 在测试执行过程中，观察到部分用例在 Android 16 模拟器环境下出现执行失败。失败的主要原因为模拟器在持续运行一段时间后，系统响应速度明显下降，界面渲染出现延迟，目标元素虽已存在于应用页面但尚未完成布局加载，脚本在此时尝试定位元素时抛出 `NoSuchElementException`。值得注意的是，相同的测试用例在 Android 13 真机上能够稳定通过，说明此类失败并非应用逻辑本身存在缺陷，而是由测试环境的性能波动所引发的。`runStep` 方法会在捕获该异常后，将报错类型、发生步骤及异常信息自动写入日志，方便测试人员快速定位问题所在，无需重新运行整个测试集即可判断失败原因。图5-9展示了日志中记录的典型失败条目及对应的异常信息。

| TestTime | TestName       | StepName                   | Status   | Duration(ms) | ErrorPoint                        | ErrorContent                                                                          | PassedCases | TotalCases | SuccessRate(%) |
|----------|----------------|----------------------------|----------|--------------|-----------------------------------|---------------------------------------------------------------------------------------|-------------|------------|----------------|
| #####    | LoginTest      | Click Views                | PASS     | 1735         |                                   |                                                                                       | 1           | 1          | 100            |
| #####    | LoginTest      | Click TextFields           | PASS     | 6252         |                                   |                                                                                       | 2           | 2          | 100            |
| #####    | LoginTest      | Input TextFields           | PASS     | 2743         |                                   |                                                                                       | 3           | 3          | 100            |
| #####    | RadioGroupTest | Back To Main Page          | PASS     | 2236         |                                   |                                                                                       | 4           | 4          | 100            |
| #####    | RadioGroupTest | Click Views                | PASS     | 1804         |                                   |                                                                                       | 5           | 5          | 100            |
| #####    | RadioGroupTest | Click Radio Group          | PASS     | 4512         |                                   |                                                                                       | 6           | 6          | 100            |
| #####    | RadioGroupTest | Clear Button               | PASS     | 741          |                                   |                                                                                       | 7           | 7          | 100            |
| #####    | RadioGroupTest | Snack Button               | PASS     | 741          |                                   |                                                                                       | 8           | 8          | 100            |
| #####    | RadioGroupTest | Breakfast Button           | PASS     | 744          |                                   |                                                                                       | 9           | 9          | 100            |
| #####    | RadioGroupTest | Lunch Button               | PASS     | 743          |                                   |                                                                                       | 10          | 10         | 100            |
| #####    | RadioGroupTest | Dinner Button              | PASS     | 737          |                                   |                                                                                       | 11          | 11         | 100            |
| #####    | RadioGroupTest | Back To Main Page          | PASS     | 1748         |                                   |                                                                                       | 12          | 12         | 100            |
| #####    | ActivityTest   | Click App                  | PASS     | 1802         |                                   |                                                                                       | 13          | 13         | 100            |
| #####    | ActivityTest   | Click Activity             | PASS     | 1349         |                                   |                                                                                       | 14          | 14         | 100            |
| #####    | ActivityTest   | Click Custom Title         | PASS     | 1368         |                                   |                                                                                       | 15          | 15         | 100            |
| #####    | ActivityTest   | Change Left Title          | PASS     | 1777         |                                   |                                                                                       | 16          | 16         | 100            |
| #####    | ActivityTest   | Change Right Title         | PASS     | 1776         |                                   |                                                                                       | 17          | 17         | 100            |
| #####    | ActivityTest   | Back From Custom Title     | PASS     | 75           |                                   |                                                                                       | 18          | 18         | 100            |
| #####    | ActivityTest   | Hello World                | PASS     | 1941         |                                   |                                                                                       | 19          | 19         | 100            |
| #####    | ActivityTest   | Back From Hello World      | PASS     | 622          |                                   |                                                                                       | 20          | 20         | 100            |
| #####    | ActivityTest   | Click Receive Result       | PASS     | 1791         |                                   |                                                                                       | 21          | 21         | 100            |
| #####    | ActivityTest   | Click Get Result For Corky | PASS     | 86           |                                   |                                                                                       | 22          | 22         | 100            |
| #####    | ActivityTest   | Corky Button               | ERROR    | 73           | Corky result check                | NoSuchElementException: An element could not be located on the page using the given s | 22          | 23         | 95.65          |
| #####    | Main           | Unexpected Error           | ERROR    | 47375        | Main Execution                    | RuntimeException: Element not found. Test stopped at DirectConstructorHandleAccessor. | 22          | 24         | 91.67          |
| #####    | Summary        | All Test Cases             | FINISHED | 47376        | Total Passed: 22, Total Cases: 24 |                                                                                       | 22          | 24         | 91.67          |
| #####    | Final Error    | Element Not Found          | STOPPED  | 47376        | NoSuchElementException            | Element not found. Test stopped at DirectConstructorHandleAccessor.java:62            | 22          | 24         | 91.67          |

图 5-9 综合测试失败时的 test\_log(2) 执行结果（模拟器）

### 5.3.4 SauceLabs My Demo App

为了进一步验证自动化测试框架对不同种类移动应用的适配性，本次研究也选了 SauceLabs 发布的 My Demo App 当作第二组测试对象。相比于 ApiDemos，My Demo App 提供了更接近真实业务场景的功能模块，包括购物流程（Shopping）、登录验证（Login）、Web 页面搜索（Website）、摄像头操作、指纹模拟等。选择 My Demo App 的主要动机在于：

- 业务场景丰富：能够同时覆盖多种用户操作流程，适合验证框架在连续多步骤和多模块交互下的稳定性。
- 功能多样性：除基础界面操作外，还包含 WebView 页面交互和购物流程模拟，可以测试框架在不同控件和组件上的适用性。
- 跨平台兼容性：SauceLabs 提供的应用能够在多种 Android 模拟器和真实设备上运行，便于评估框架在不同分辨率和系统版本环境下的兼容性。

本次研究针对 My Demo App 设计的测试用例主要围绕三大模块：购物流程、登录功能以及 WebView 页面搜索。虽然这些测试用例与第三章针对 ApiDemos 设计的 F01-C03 测试用例属于不同应用，但仍遵循同样的模块化封装和断言设计原则，保证框架在不同应用上的可复用性和稳定性。通过引入 My Demo App，本文不仅验证了框架在单一功能测试上的可靠性，也评估了其在复杂业务逻辑与连续操作场景下的执行能力。

测试脚本以 Appium + UiAutomator2 编写，采用模块化封装思路和 Page Object 模式，实现各模块操作的规范化管控。以下伪代码展示了框架在 My Demo App 上的核心运行逻辑。

---

**Algorithm 4** Pseudocode for My Demo App

---

**Data:** Mobile application My Demo App, real device/emulator

**Result:** End-to-end shopping process and WebView test executed successfully

- 1 Initialize desired capabilities: device name, platform version, automation engine, app package and activity Connect to Appium server at localhost:4723 Launch My Demo App using AndroidDriver
  - 2 Start MyDemoAppTest and record the start time
  - 3 Locate the first product image "Backpack" and click it
  - 4 Locate the five-star rating element and click it
  - 5 Locate the increase quantity button and click it
  - 6 Locate the "Add to Cart" button and click it
  - 7 Locate the cart icon and click it
  - 8 Click the "Proceed to Checkout" button
  - 9 Wait for the login page to load Input username and password Click the login button
  - 10 Input shipping address information: Click the button to save user information for checkout
  - 11 Input payment information: Click the button to save payment information and enter the review order page
  - 12 Click the button to complete checkout Locate the button "Tap to open catalog" and click it If the checkout process is completed successfully, print "Checkout Success"
  - 13 Locate the "WebView" menu item and click it Input the URL "https://cs.nju.edu.cn"
  - 14 Scroll the WebView page until the text " 版权所有 © 南京大学计算机学院" is visible Check whether the page text contains " 南京大学计算机学院" If the expected text is found, print "Correct Website" Otherwise, print "Wrong Website"
  - 15 Record the total execution time, error point, error content and success rate in the CSV log file
  - 16 Print "MyDemoAppTest Finished" Quit the driver
- 

在执行过程中，框架通过 `runStep` 方法统一管理每个操作步骤，捕获执行异常并将日志结构化输出。每条操作的成功状态、执行耗时、异常信息均被记录到

CSV 文件中，便于后续分析和结果追踪。

综合以上测试，本文得出以下结论：

- 跨模块连续执行能力：My Demo App 的测试表明，框架能够同时处理多个功能模块的连续操作，包括购物、登录和 Web 页面交互。
- 日志与断言可追溯性：runStep 方法提供的结构化日志和断言结果记录，保证了测试执行的可追溯性，即便出现失败用例也可以快速定位问题。
- 适应复杂业务场景：框架在面对不同类型控件、页面元素和业务逻辑时，能够保持高复用性和稳定性，为后续扩展到其他应用提供了实践基础。
- 多设备适配验证：通过在模拟器和真实设备上的执行对比，可以发现环境差异对执行结果的影响，并体现了框架对不同 Android 系统版本的兼容能力。

总之，通过 My Demo App 的补充测试，不仅验证了框架在复杂应用场景下的适用性，还进一步展示了自动化测试脚本在跨模块、多页面和多设备环境下的可复用性与可靠性，为第五章的整体测试结果分析提供了更丰富的数据支撑，并为未来在更多应用场景下推广该自动化框架奠定了基础。

### 5.3.5 Monkey 随机稳定性测试

为了增加测试方法的多样性，本文在原有功能测试之外，补充使用 Android SDK 自带的 ADB Monkey 工具对 ApiDemos 6.0.10 进行随机稳定性测试。ADB Monkey 可以通过命令行向指定应用发送大量随机事件，包括点击、滑动、按键、Activity 切换等操作，并通过 -v 参数输出测试执行日志。与按照固定步骤执行的测试不同，Monkey 测试更强调随机性和稳定性观察，主要用于检查应用在大量随机操作下是否会出现崩溃、无响应或异常退出等问题。

本节选择在 Android 16 模拟器环境下执行 Monkey 测试。没有选择真实设备作为测试环境，主要是因为 Monkey 随机事件具有不可控性，可能触发系统返回、应用切换、文件访问或其他系统级操作。真实设备中通常保存有个人文件、账号信息和其他常用应用，如果直接执行大量随机事件，可能影响真实设备的正常使用状态。相比之下，模拟器环境更加独立和可控，测试前后也可以快速恢复初始状态，因此更适合进行随机事件测试。

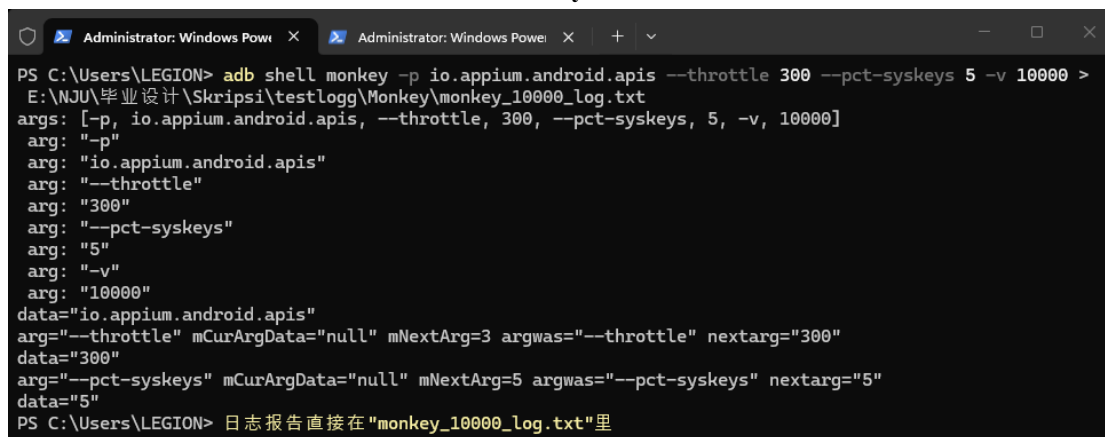
本文使用的测试命令如下：

**adb shell monkey -p io.appium.android.apis --throttle 300 --pct-syskeys 5 -v 10000**

其中，io.appium.android.apis 为 ApiDemos 6.0.10 的应用包名；--throttle 300 表示每两个随机事件之间间隔 300 毫秒，避免事件发送过快导致测试结果不稳定；--pct-syskeys 5 表示保留 5% 的系统按键事件，使测试过程具有一定随机性；-v 表示输出详细日志；10000 表示本次测试共发送 10000 个随机事件。命令最后通过重定向符号 > 将执行结果保存到本地日志文件 monkey\_10000\_log.txt 中，便于后续查看和分析。

在测试过程中，ADB Monkey 会自动启动指定应用，并按照随机事件序列持续向应用发送操作。-v 日志中会记录测试参数、随机种子、允许测试的包名、事件比例、Activity 启动信息、事件发送进度以及最终执行状态。例如日志中会显示:AllowPackage: io.appium.android.apis，表示测试对象被限制为 ApiDemos；// Sending event #1000 等内容表示随机事件执行进度；最后如果出现 Events injected: 10000 和 // Monkey finished，则说明本轮随机事件测试已经完整执行。

图 5-10 ADB Monkey 命令行执行示意图



```
PS C:\Users\LEGION> adb shell monkey -p io.appium.android.apis --throttle 300 --pct-syskeys 5 -v 10000 >
E:\NJU\毕业设计\Skripsi\testlogg\Monkey\monkey_10000_log.txt
args: [-p, io.appium.android.apis, --throttle, 300, --pct-syskeys, 5, -v, 10000]
arg: "-p"
arg: "io.appium.android.apis"
arg: "--throttle"
arg: "300"
arg: "--pct-syskeys"
arg: "5"
arg: "-v"
arg: "10000"
data="io.appium.android.apis"
arg="--throttle" mCurArgData="null" mNextArg=3 argwas="--throttle" nextarg="300"
data="300"
arg="--pct-syskeys" mCurArgData="null" mNextArg=5 argwas="--pct-syskeys" nextarg="5"
data="5"
PS C:\Users\LEGION> 日志报告直接在 "monkey_10000_log.txt" 里
```

图 5-11 ADB Monkey 测试日志片段

```
:Sending Trackball (ACTION_MOVE): 0:(-1.0,-5.0)
:Sending Trackball (ACTION_MOVE): 0:(-1.0,2.0)
// Rejecting start of Intent { act=android.intent.action.SHOW_ALARMS cmp=com.google.android.deskclock/com.android.deskclock.HandleApiCalls } in package com.google.android.deskclock
:Sending Trackball (ACTION_MOVE): 0:(0.0,-1.0)
:Sending Trackball (ACTION_MOVE): 0:(-2.0,4.0)
:Sending Trackball (ACTION_MOVE): 0:(4.0,1.0)
:Sending Trackball (ACTION_MOVE): 0:(3.0,-1.0)
//[calendar_time:2026-06-05 02:51:23.304 system_uptime:86029913]
// Sending event #9700
:Sending Trackball (ACTION_UP): 0:(0.0,0.0)
:Sending Trackball (ACTION_MOVE): 0:(-1.0,-1.0)
:Sending Trackball (ACTION_MOVE): 0:(0.0,2.0)
:Sending Trackball (ACTION_MOVE): 0:(-4.0,1.0)
:Sending Trackball (ACTION_MOVE): 0:(-3.0,-3.0)
Switch: #Intent;action=android.intent.action.MAIN;category=android.intent.category.LAUNCHER;launchFlags=0x10200000;component=io.apium.android.apis/.ApiDemos;end
// Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=io.apium.android.apis/.ApiDemos } in package io.apium.android.apis
:Sending Trackball (ACTION_MOVE): 0:(0.0,0.0)
//[calendar_time:2026-06-05 02:51:29.671 system_uptime:86036191]
// Sending event #9800
// activityResuming(io.apium.android.apis)
// Allowing start of Intent { cmp=io.apium.android.apis/.animation.AnimationSeeking } in package io.apium.android.apis
:Sending Trackball (ACTION_MOVE): 0:(0.0,4.0)
:Sending Trackball (ACTION_MOVE): 0:(2.0,3.0)
:Sending Trackball (ACTION_MOVE): 0:(-4.0,-1.0)
:Sending Trackball (ACTION_MOVE): 0:(3.0,3.0)
//[calendar_time:2026-06-05 02:51:35.330 system_uptime:86041849]
// Sending event #9900
:Sending Trackball (ACTION_MOVE): 0:(-2.0,4.0)
:Sending Flip keyboardOpen=false
Got IOException performing flip:java.io.FileNotFoundException: /dev/input/event0: open failed: EACCESS (Permission denied)
// Injection failed
:Sending Trackball (ACTION_MOVE): 0:(-2.0,-3.0)
:Sending Trackball (ACTION_MOVE): 0:(4.0,2.0)
:Sending Trackball (ACTION_MOVE): 0:(-1.0,-5.0)
:Sending Trackball (ACTION_MOVE): 0:(1.0,2.0)
Events injected: 10000
:Sending rotation degree=0, persist=false
:Dropped: keys=0 pointers=0 trackballs=0 flips=31 rotations=0
## Network stats: elapsed time=771270ms (0ms mobile, 0ms wifi, 771270ms not connected)
// Monkey finished
```

需要说明的是，在 Monkey 测试过程中，应用界面可能会因为随机事件出现返回主界面、切换页面或短暂离开当前 Activity 的情况。这属于 Monkey 随机测试的正常表现，并不一定代表应用存在功能错误。因此，本文在后续结果分析中主要关注日志中是否出现 CRASH、ANR、FATAL EXCEPTION、异常退出等关键信息，并结合日志末尾的事件完成情况判断应用在随机操作下的稳定性。

## 5.4 测试结果分析

### 5.4.1 综合测试结果分析

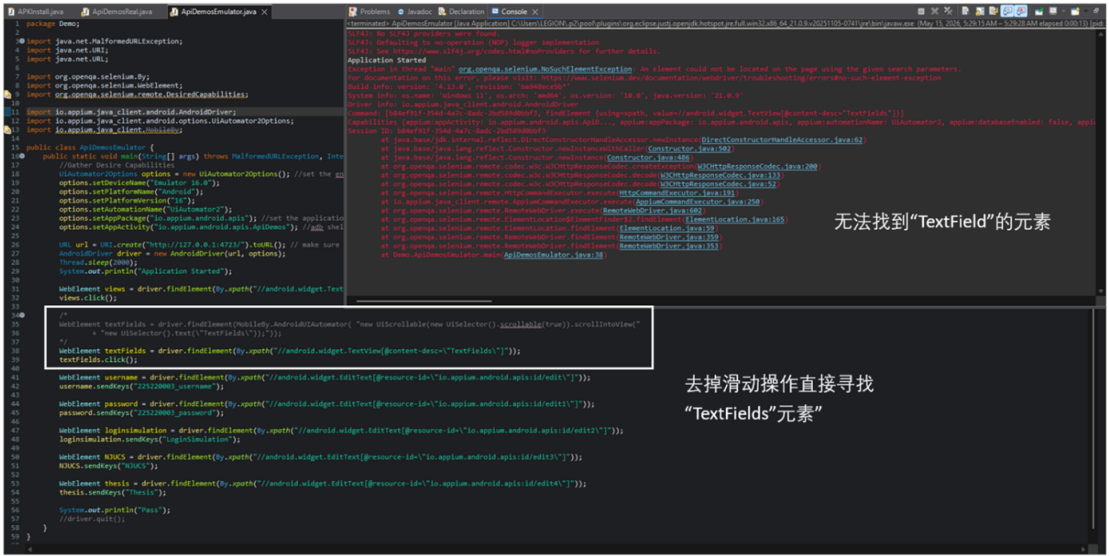
基于 ApiDemos 6.0.10 的登录模拟功能所设计的自动化测试脚本成功覆盖了核心功能、页面跳转、输入操作以及界面元素的交互操作。总体来看，测试脚本能够顺利执行绝大部分用例，输入框内容、按钮点击及页面跳转均按照预期运行，应用界面未出现崩溃、卡顿或响应异常情况，保证了功能的完整性和操作流程的连续性。

在多子场景测试用例中，本文采用计数器 sum 对断言通过情况进行统计。以表单与元素可见性测试（为例，脚本在每个单选按钮操作后执行断言比对，若结果与预期值一致则将 sum 累加，最终通过判断 sum 的值输出整体测试结论。这种方式能够在单次脚本执行中批量验证多个子场景，避免逐一手动检查，同时为后续结果汇总与统计分析提供量化依据。当前阶段 Clear、Snack 与 Breakfast 三个子用例断言完整通过（sum=3）；Lunch 与 Dinner 的操作步骤均正常执行，断言

逻辑因调试需要临时屏蔽，待预期值确认后将补全，届时 sum 上限将更新为 4。

在测试过程中，少数用例出现了执行失败的情况。主要原因在于部分目标元素初始未显示在屏幕视图内，直接定位时无法找到元素。为解决这一问题，本文在脚本中引入了 UiScrollable 滑动查找方法，使屏幕自动滑动至目标元素所在位置后再进行操作，从而有效避免了因屏幕显示限制导致的失败用例。这一做法验证了框架对复杂界面和动态布局的适应能力，也体现了自动化测试在处理多种屏幕和设备环境下的灵活性。

图 5-12 当元素不在当前屏幕内失败结果过用例



除上述元素定位问题外，移动应用测试还面临多种运行时中断场景，这类中断往往是导致自动化用例失败的潜在原因之一。常见的中断类型包括：来电与短信通知、设备充电或断开连接、闹钟与系统通知弹窗、网络断开与恢复、网络切换（如 Wi-Fi 与移动数据之间的切换）、低电量或电量充满提示、设备关机，以及用户点击外部链接跳出应用等<sup>[22]</sup>。这些中断事件可能在脚本执行期间触发系统级弹窗或强制切换前台页面，导致目标元素不可达或操作流程中断，从而引发测试失败。

在性能和操作流畅度方面，测试过程中观察到页面滑动、按钮点击和输入操作均较为顺畅，响应时间在可接受范围内，无明显延迟或卡顿现象。虽然本文的框架并未专门引入性能测量工具，但通过重复执行典型场景和滑动操作，可以初步判断脚本执行效率和交互流畅度，进一步说明框架在自动化测试过程中具备较好的操作体验和可重复性。

此外，框架在脚本复用性和稳定性方面表现良好。相同的测试用例可以在不同 Android 设备、不同屏幕尺寸及操作系统版本下重复执行，无需额外修改，大大提高了自动化测试的效率和可维护性。同时，结合日志记录和报告模块，可以清晰地记录每次测试执行结果，包括成功用例、失败用例及操作步骤，为后续回归测试、问题分析以及框架优化提供了可靠依据。综上所述，本文的自动化测试框架不仅验证了核心功能的正确性，也体现了在多设备、多版本环境下的适应能力和可扩展性，为移动应用自动化测试提供了实践价值和理论参考。

## 5.4.2 综合测试结果分析

三个测试模块全部执行之后，这一节借助 `runStep` 机制自动生成的 CSV 格式日志文件，把所有测试结果都整理了一遍。跟 5.4.1 节单独讲单个模块不同，这一节以日志收集到的量化数据为核心，对比真机跟模拟器运行效率的差别，同时结合执行失败的日志记录，深入分析测试过程里出现的异常问题。

这次用到的综合测试集由 `LoginTest`、`RadioGroupTest`、`ActivityTest` 三类用例组成，总共有 30 个测试步骤，分别在装了 Android 13 系统的真机（对应日志标识 `test_log(3)`）和装了 Android 16 系统的模拟器（对应日志标识 `test_log(4)`）上运行完成。两次测试的 30 个步骤全部都跑通了，成功率都是 100%，两份日志的总运行时长放在一起对比，发现两种测试环境的响应速度差得挺多。

真机环境下跑完所有测试总共花了 112690ms，换算下来大概是 112 秒，相同的测试内容在模拟器环境只花了 64468ms，大概 64 秒，模拟器的运行速度差不多是真机的 1.75 倍，把对比细化到单个测试步骤来看，两种环境的差距在 `ActivityTest` 测试阶段最明显：拿 `Corky Button` 测试步骤举例，真机跑要花 7882ms，模拟器只需要 1754ms；到 `Violet Button` 步骤的时候，真机用时 8040ms，模拟器只用时 1184ms，出现这种差距主要是因为，真机完成跨 `Activity` 页面跳转时，得等实体硬件走完系统渲染的流程，而模拟器直接在本地设备运行，没有硬件层面的通信延迟，所以响应速度更快。两种环境下各模块典型步骤的耗时对比，可看图 5-7 和图 5-8。

为了进一步看测试框架在不稳定运行环境里的适配效果，本文挑了一份有执行失败记录的运行日志（对应图 5-9）做专门分析。这次测试是在模拟器环境里做的，一共记录了 24 个测试步骤的运行数据，其中 22 个步骤顺利通过，2 个



步骤执行失败，最终测试成功率是 91.67%。

从日志记录里能看到，失败出现在 ActivityTest 模块的 Corky Button 步骤里，日志里的 ErrorContent 字段完整记下了异常内容：NoSuchElementException: An element could not be located on the page，要找的目标是 //android.widget. Button[@content-desc= “Corky” ]。结合执行时的时间戳分析，Click Get Result For Corky 步骤执行完且通过，只花了 86ms，而紧跟在它后面的 Corky Button 步骤也被触发，才耗时 73ms 就抛出了异常，这说明脚本在点完 Get Result 按钮之后，几乎马上就尝试定位 Corky 按钮，但这时候目标 Activity 页面还没渲染完，Corky 按钮还没显示出来，最后导致定位失败。

本次运行出问题，主要原因可以归成三点：第一点是模拟器开久了，系统反应变慢，页面跳转的速度赶不上脚本运行的节奏；第二点是现在用的脚本没有设置明确的等待规则，最后就导致脚本运行的时间点和界面渲染进度对不上，出问题之后，runStep 函数会先把错误信息抓出来写到日志文件里，接着往上抛出 RuntimeException 异常，到最后 Main 模块就会输出 “Element not found. Test stopped” 的提示，直接停下整个测试流程，后面所有测试步骤都没法继续走下去了。图5-9里存着这次出错步骤对应的完整日志内容。

这个失败例子也暴露了自动化测试里一类很典型的、依赖环境的问题：脚本本身逻辑没有错，被测的应用功能也都正常，但测试环境的性能出现了随机波动，页面响应的时间比脚本预先设置的隐式等待阈值要长，最后导致找不到对应元素。针对这类问题，通用的解决办法是在核心操作步骤前面加上明确的等待逻辑，比如调用 TimeUnit 的相关方法，让脚本确定目标元素已经能交互了，再发起下一步操作，这样就能让测试流程在性能不同的环境里跑的时候更稳定，结合测试全通过的对照结果能看出来，这次问题是环境性能波动带来的偶然异常，不是测试框架设计有问题，也不是应用本身逻辑错了。

三次运行的日志数据统计整合之后发现，本文提出的自动化测试框架，在性能稳定的运行环境里能做到 30 个测试全过，也就是通过率 100%，配套的 CSV 日志机制在出异常的时候，能完整记下错误类型、触发步骤和时间戳信息，给排查问题和之后优化功能给出足够的数据支持，这次的测试结果也给5.4.3节手动测试和自动化测试的对比分析提供了量化的依据。

### 5.4.3 手动测试与自动化测试比较

为能进一步衡量自动化测试框架实际用起来的价值，本章节把测试执行时长当成主要指标。自动化和人工测试两种模式的横向对比，结合真机、模拟器两类设备的好几轮运行数据，分析不一样的测试方案和运行设备条件下的效率差距。

围绕执行时长这个维度，研究选了一模一样的测试用例集合，分别在真机、模拟器两种场景做多轮测试，再和测试人员手动一步步走完同场景测试花掉的时间做对比。自动化测试这边。结果表明，模拟器环境里两轮跑下来耗时分别是 64468ms 和 65885ms，平均大概 65 秒；真机场景下前两轮执行花的时间分别为 112690ms 与 99996ms，平均耗时大概 106 秒，把后几轮测试的运行表现合起来看，模拟器运行时长稳定在 60 到 70 秒这个区间，真机的稳定运行区间则在 95 到 126 秒之间。人工测试的时候，测试人员已经完全摸清楚应用架构和操作路径了，按顺序做完全部 30 项测试步骤，实际花掉的时间大概是 189 秒（也就是 3 分钟多）。

表 5-1 兼容性与性能测试模块

| 测试方式  | 执行环境           | 耗时范围      | 平均耗时    |
|-------|----------------|-----------|---------|
| 自动化测试 | 模拟器 Android 16 | 64-66 秒   | 约 65 秒  |
| 自动化测试 | 真机 Android 13  | 100-113 秒 | 约 106 秒 |
| 手动测试  | 真机 Android 13  | -         | 约 189 秒 |

结合上面这些统计结果能看出来，模拟器搭配自动化测试的效率表现最佳，运行时长约只有人工测试的三分之一。真机自动化测试的效率次之，运行的时间大概是人工测试的二分之一。即便是响应速度偏慢的真机运行环境，自动化测试的执行速度还是明显比人工操作快，而且每轮测试的步骤先后顺序、输入参数和校验逻辑全都是统一的，不会受到测试人员操作习惯或者注意力不集中这类人为因素的影响。

另外一点，用完全一样的测试脚本在两类环境里运行，真机的平均消耗时长大概是模拟器的 1.6 倍。主要由三方面核心因素一起造成：

1. 通信链路差异：模拟器还有 AppiumServer 都被部署在同一台主机上，测试脚本通过本地回环地址 localhost 完成交互，数据传输产生的延迟基本可以

忽略。而真机测试的时候，是用 USB 方式连接到主机完成数据交互，所有的 ADB 命令都得经过物理接口转发，这里存在没法消除的硬件层面的传输延迟，如果测试用例里包含了比较多操作步骤，那这种延迟积累起来影响会变得很明显。

2. 厂商系统优化与权限限制：本次测试用到的实体设备，是型号为 OPPO CPH2025、运行 Android13 系统的手机，它预先装好了设备厂商做了深度定制的操作系统，系统自带后台能耗管控、权限分级调控还有安全防护策略这类多项内置机制。这些系统自带的机制有可能会让 UiAutomator2 接收到的操作指令，额外多出一部分运算处理的成本，比较典型的场景就有权限合规校验、操作行为拦截核验这些环节，最终就会导致单步指令的执行耗时，一般都比模拟器环境要高。拿 CorkyButton 测试步骤的运行数据举例，子实体设备的执行耗时为 7882ms，相同条件下模拟器只需要 1754ms，两者的差值能达到 4.5 倍，这个差距足够说明厂商定制系统带来的性能损耗，这种损耗在跨 Activity 跳转这类操作场景里，表现得尤其明显。
3. 模拟器环境更为纯净：这个模拟器用的是原生 Android（AOSP）系统镜像，没有加入厂商做的定制系统层级，也没有后台推送通知、活跃进程占资源这类干扰，所有系统资源都可以优先给测试任务用。不一样的是，真机跑测试程序的时候，很容易被系统弹窗通知、后台留着的进程、屏幕亮度调节这类变量影响，最后会让响应时长出现没规律的上下浮动。

虽然这类性能波动的问题，真机测试在还原用户日常使用场景这块的作用，还是没法被其他方案替代。真机能精准给出厂商定制系统对应的实际渲染效果、软硬件交互反馈，还有真实网络环境里的应用运行状态，这些变量都没法靠模拟器做到高精度还原。按照这个特点，制定实际测试方案的时候，可以先把模拟器当成快速核验功能、做回归测试的主要运行环境，把真机测试当成版本发布前检查兼容性和稳定性的辅助方法，靠两类环境配合使用，达成测试执行效率和结果可信度的最优平衡。

针对结果一致、可重复做出来的需求，自动化测试也有很突出的好处。人工做测试的过程高度依赖测试人员的操作记忆和主观判断，重复跑相同流程的时候容易出现步骤漏做、输入参数不一样的问题；自动化测试脚本却能保证每一轮执行的操作流程、输入参数和判断逻辑完全统一，再配上 CSV 格式运行日志的

自动存储功能，每一次测试的结果都能被追溯、拿来横向比对。这一特性的主要作用特别明显：应用更新完功能迭代之后，自动化测试套件只需要几分钟就能覆盖全部功能点，做完校验换成测试人员手动反复做同样场景的测试，耗时会涨到好几倍，而且很难保证每一轮操作都严格按同一个标准来做。

本次研究提出的自动化测试框架在运行效率、结果稳定性和问题追溯能力这三项指标上，表现全都明显比人工测试更好，充分说明搭建体系化的自动化测试框架确实有实际用处，同时也给第六章做研究总结、梳理未来方向给出了扎实的实验数据支撑。

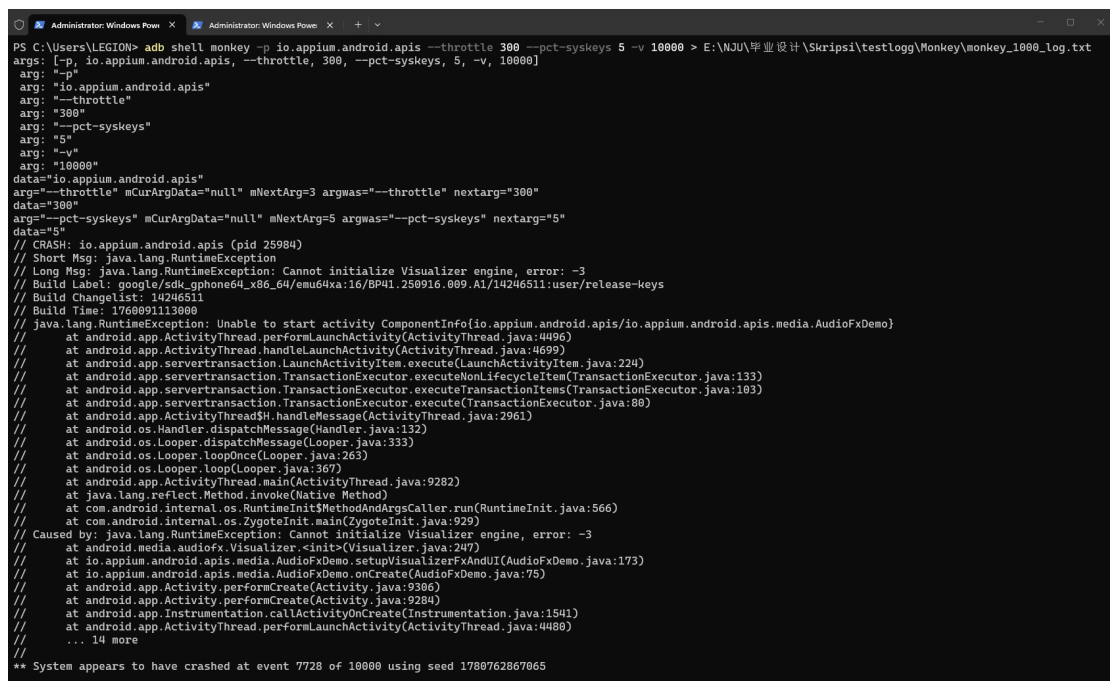
#### 5.4.4 Monkey 测试结果分析

本节对 5.3.5 中执行的 ADB Monkey 随机稳定性测试结果进行分析。与前文基于 Appium 的固定流程测试不同，Monkey 测试并不按照预先设计好的测试步骤执行，而是通过大量随机事件观察应用在非固定操作下的运行状态。因此，Monkey 测试结果的分析重点不在于某个具体页面是否按预期跳转，而在于应用是否出现崩溃、无响应、异常退出，以及日志中是否记录了严重异常信息。

从完整执行的 Monkey 日志来看，本次测试共向 ApiDemos 6.0.10 发送了 10000 个随机事件。日志中出现了“Events injected: 10000”和“Monkey finished”等信息，说明该轮随机事件测试已经完整执行结束。从事件内容来看，Monkey 在测试过程中持续发送 Trackball 移动、Activity 切换、系统按键等随机事件，并多次启动或返回到 ApiDemos 的不同页面。测试过程中虽然出现了部分系统级事件被拒绝的情况，例如尝试启动非目标应用或系统组件时被拒绝，但由于命令中指定了应用包名，随机测试的主要范围仍然集中在 ApiDemos 应用之内。

在完整执行的日志中，还可以看到“Got IOException performing flip”和“Permission denied”等信息。这类信息主要出现在 Monkey 尝试执行键盘翻转或输入设备相关事件时，系统因权限限制无法访问相关设备节点。该情况属于 Android 模拟器环境中的系统权限限制，并不直接说明 ApiDemos 应用本身存在功能错误。从日志结果来看，该类事件并未导致测试中断，Monkey 最终仍然完成了 10000 个随机事件注入。因此，在稳定执行的测试轮次中，ApiDemos 在大量随机操作下没有出现明显的崩溃或无响应现象，说明应用在普通随机操作场景下具有一定稳定性。

但是, 在另一轮 Monkey 测试日志中, 随机事件执行到第 7728 个事件时出现了应用崩溃。日志显示崩溃对象为 `io.appium.android.apis`, 异常类型为 `java.lang.RuntimeException`, 具体原因是 `Cannot initialize Visualizer engine, error: -3`。从崩溃堆栈可以看到, 该异常发生在 `io.appium.android.apis.media.AudioFxDemo` 页面启动过程中, 与音频可视化 `Visualizer` 引擎初始化失败有关。这说明 Monkey 随机事件进入了 `ApiDemos` 中的媒体相关页面, 并触发了固定流程测试中没有覆盖到的功能模块。由于模拟器环境对部分音频或硬件能力的支持可能不完整, 因此该异常更可能与模拟器环境下的音频可视化功能支持不足有关, 但它仍然说明该页面在随机测试和特定环境下存在稳定性风险。



```
PS C:\Users\LEGION> adb shell monkey -p io.appium.android.apis --throttle 300 --pct-syskeys 5 -v 10000 > E:\NJU\毕业设计\5kripsi\testlogg\Monkey\monkey_1000_log.txt
args: [-p, io.appium.android.apis, --throttle, 300, --pct-syskeys, 5, -v, 10000]
arg: "-p"
arg: "io.appium.android.apis"
arg: "--throttle"
arg: "300"
arg: "--pct-syskeys"
arg: "5"
arg: "-v"
arg: "10000"
data="io.appium.android.apis"
args="--throttle" mCurArgData="null" mNextArg=3 argwas="--throttle" nextarg="300"
data="300"
args="--pct-syskeys" mCurArgData="null" mNextArg=5 argwas="--pct-syskeys" nextarg="5"
data="5"
// CRASH: io.appium.android.apis (pid 25984)
// Short Msg: java.lang.RuntimeException
// Long Msg: java.lang.RuntimeException: Cannot initialize Visualizer engine, error: -3
// Build Label: google/sdk_gphone64_x86_64/emu64xa:16/BP41.250916.009.A1/14246511:user/release-keys
// Build Changelist: 14246511
// Build Time: 1760891113000
// java.lang.RuntimeException: Unable to start activity ComponentInfo{io.appium.android.apis/io.appium.android.apis.media.AudioFxDemo}
// at android.app.ActivityThread.performLaunchActivity(ActivityThread.java:4496)
// at android.app.ActivityThread.handleLaunchActivity(ActivityThread.java:4699)
// at android.app.servertransaction.LaunchActivityItem.execute(LaunchActivityItem.java:224)
// at android.app.servertransaction.TransactionExecutor.executeLifecycleItem(TransactionExecutor.java:133)
// at android.app.servertransaction.TransactionExecutor.executeTransactionItems(TransactionExecutor.java:183)
// at android.app.servertransaction.TransactionExecutor.execute(TransactionExecutor.java:80)
// at android.app.ActivityThread$H.handleMessage(ActivityThread.java:2961)
// at android.os.Handler.dispatchMessage(Handler.java:132)
// at android.os.Looper.dispatchMessage(Looper.java:333)
// at android.os.Looper.loopOnce(Looper.java:263)
// at android.os.Looper.loop(Looper.java:367)
// at android.app.ActivityThread.main(ActivityThread.java:9282)
// at java.lang.reflect.Method.invoke(Native Method)
// at com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:566)
// at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:929)
// Caused by: java.lang.RuntimeException: Cannot initialize Visualizer engine, error: -3
// at android.media.audiofx.Visualizer.<init>(Visualizer.java:247)
// at io.appium.android.apis.media.AudioFxDemo.setupVisualizerFXAndUI(AudioFxDemo.java:173)
// at io.appium.android.apis.media.AudioFxDemo.onCreate(AudioFxDemo.java:75)
// at android.app.Activity.performCreate(Activity.java:9306)
// at android.app.Activity.performCreate(Activity.java:9284)
// at android.app.Instrumentation.callActivityOnCreate(Instrumentation.java:1541)
// at android.app.ActivityThread.performLaunchActivity(ActivityThread.java:4480)
// ... 14 more
** System appears to have crashed at event 7728 of 10000 using seed 1780762867065
```

图 5-13 ADB Monkey 测试崩溃信息输出

通过对比两类日志可以发现, Monkey 测试具有较强的不确定性。同样是 10000 个随机事件测试, 有的轮次可以完整执行结束, 有的轮次则可能因为随机进入某些特殊功能页面而触发异常。这正是 Monkey 测试的特点和价值所在: 它能够通过随机事件覆盖人工设计用例不一定会主动触达的页面和操作路径, 从而发现固定流程测试难以暴露的潜在问题。

从测试结果来看, ADB Monkey 测试对本文原有测试方法形成了有效补充。前文的登录测试、表单测试和 `ActivityTest` 主要验证固定操作路径下功能是否正常, 而 Monkey 测试进一步观察应用在随机点击、随机切换和系统按键事件下的

```

// Rejecting start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] pkg=com.google.android.youtube cmp=com.google.android.youtube/.app.honeycomb.Shell
$HomeActivity } in package com.google.android.youtube
// Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] pkg=io.appium.android.apis cmp=io.appium.android.apis/.ApiDemos } in package
io.appium.android.apis
:Sending Trackball (ACTION_MOVE): 0:(0.0,1.0)
// Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.HOME] cmp=com.google.android.apps.nexuslauncher/.NexusLauncherActivity } in package
com.google.android.apps.nexuslauncher
// activityResuming(io.appium.android.apis)
// Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] pkg=io.appium.android.apis cmp=io.appium.android.apis/.ApiDemos } in package
io.appium.android.apis
// Allowing start of Intent { cmp=io.appium.android.apis/.ApiDemos } in package io.appium.android.apis
// Allowing start of Intent { cmp=io.appium.android.apis/.accessibility.AccessibilityNodeProviderActivity } in package io.appium.android.apis
:Sending Trackball (ACTION_MOVE): 0:(-4.0,2.0)
:Sending Trackball (ACTION_MOVE): 0:(-5.0,-1.0)
// activityResuming(io.appium.android.apis)
// activityResuming(io.appium.android.apis)
// [calendar_time:2026-06-05 02:38:03.123 system_uptime:85229643]
// Sending event #2600
:Sending Trackball (ACTION_MOVE): 0:(-2.0,-2.0)
// Allowing start of Intent { cmp=io.appium.android.apis/.ApiDemos } in package io.appium.android.apis
:Sending Trackball (ACTION_MOVE): 0:(-3.0,0.0)
:Sending Trackball (ACTION_MOVE): 0:(-5.0,2.0)
$Switch: $Intent:action=android.intent.action.MAIN;category=android.intent.category.LAUNCHER;launchFlags=0x10200000;component=io.appium.android.apis/.ApiDemos;end
// Allowing start of Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=io.appium.android.apis/.ApiDemos } in package io.appium.android.apis
:Sending Flip keyboardopen=true
Got IOException performing flip:java.io.FileNotFoundException: /dev/input/event0: open failed: EACCES (Permission denied)
// Injection failed
// activityResuming(io.appium.android.apis)
:Sending Trackball (ACTION_MOVE): 0:(-4.0,0.0)
// [calendar_time:2026-06-05 02:38:09.583 system_uptime:85236102]
// Sending event #2700
:Sending Trackball (ACTION_UP): 0:(0.0,0.0)
// Allowing start of Intent { cmp=io.appium.android.apis/.ApiDemos } in package io.appium.android.apis
// activityResuming(io.appium.android.apis)
// Allowing start of Intent { cmp=io.appium.android.apis/.ApiDemos } in package io.appium.android.apis
// Allowing start of Intent { cmp=io.appium.android.apis/.media.AudioFxDemo } in package io.appium.android.apis
** Monkey aborted due to error.
Events injected: 7728
:Sending rotation degree=0, persist=false
:Dropped: keys=0 pointers=0 trackballs=0 flips=16 rotations=0
## network stats: elapsed time=562782ms (0ms mobile, 0ms wifi, 562782ms not connected)

```

图 5-14 ADB Monkey 测试崩溃日志片段

稳定性表现。尤其是 AudioFxDemo 页面触发的 RuntimeException，说明随机测试能够发现某些边缘模块或硬件相关功能在特定环境下的异常情况。

总体来看，本次 Monkey 测试结果可以总结为三点：第一，ApiDemos 在部分测试轮次中能够完成 10000 个随机事件注入，未出现明显崩溃或无响应，说明其基础界面和普通交互具有一定稳定性；第二，测试过程中出现的部分权限拒绝和系统事件失败主要与模拟器系统环境有关，并不一定代表应用功能缺陷；第三，Monkey 在随机进入媒体功能页面时发现了 Visualizer 初始化失败导致的 RuntimeException，说明随机稳定性测试能够补充固定流程测试的不足，帮助发现更隐蔽的异常场景。

因此，ADB Monkey 测试不仅增加了本文测试方法的多样性，也使测试结果分析更加完整。它证明了在手机 APP 测试中，固定流程测试和随机稳定性测试具有不同作用：固定流程测试适合验证明确功能，Monkey 测试则适合发现随机操作和边缘场景下的稳定性问题。二者结合能够提高测试覆盖范围，并为后续优化应用稳定性和完善测试方案提供参考。

## 5.5 本章小结

本章围绕第三章提出来的测试需求，还有第四章设计好的自动化测试框架，完成了 ApiDemos 6.0.10 上典型测试用例的运行和结果分析，做实验的过程里，本文利用 Appium 和 UiAutomator2 脚本控制应用，在真实设备还有模拟器上，对

登录、页面跳转、表单操作、元素能不能看见这些核心功能做了验证。测试脚本可以按照提前设定好的顺序做完所有操作，包括点击、滑动还有输入文字等等，由此验证了这个框架在功能覆盖和界面交互这两方面都是有效的。

在统计结果这块，本章加入了计数器 `sum` 来给多个子场景的断言结果做量化统计，做到了单次运行脚本的时候，就能给多个用例做批量验证，还能把汇总结果输出来，同时，做了 `ActivityTest` 综合测试，验证了多个 `Activity` 页面跳转、文本修改、结果传递还有弹窗交互这类内容，让测试场景变得更丰富。除此之外，本文还加了 `SauceLabs My Demo App` 当成补充测试对象，针对购物流程、登录还有 `Website` 搜索这些场景做了扩展测试，让测试内容不只是局限在 `ApiDemos` 这一个应用上。

在分析失败用例这部分，本章找出了几类主要的失败原因，第一类是目标元素一开始没显示在屏幕能看见的区域，针对这个情况加入了 `UiScrollable` 滑动查找的方法，让屏幕自动滚到目标元素所在的位置之后再执行操作，很好解决了因为屏幕显示范围有限导致的定位失败问题，第二类是模拟器跑了很久之后，可能会出现卡了或者页面加载变慢的情况，导致部分元素没能及时渲染出来，进而影响了脚本运行的结果。第三类是移动应用运行的时候，可能会碰到外部中断或者系统事件，比如弹出系统弹窗、切换应用、页面状态变了等等，这类情况很可能会打断脚本的运行流程，或者让目标元素找不到。

另外，本章还补充了围绕 `ADB Monkey` 做的随机稳定性测试，这个测试在 `Android 16` 模拟器环境里，给 `ApiDemos` 发送了大量随机事件，还靠 `Monkey` 自带的 `-v` 日志把运行过程记录下来。测试结果表明，有一部分测试轮次能完整跑完 10000 个随机事件，说明这个应用在普通随机操作下，是有一定稳定性的；还有一轮测试跑到第 7728 个事件的时候，触发了 `AudioFxDemo` 页面里的 `RuntimeException`，这也说明 `Monkey` 测试可以找到固定流程测试不太容易覆盖到的边缘页面，还有隐藏的稳定性问题。

在多个不同设备、不同 `Android` 版本上反复运行测试之后，这个脚本体现出了不错的复用性，运行也很稳定，实验过程里收集到的日志、自动生成的测试报告，给分析测试结果、排查问题给出了可靠的参考，同时也给之后其他功能模块的自动化测试打下了基础，本章不光验证了框架的可行性和实际能用的效果，还借助 `SauceLabs My Demo App` 和 `ADB Monkey` 测试补充了更多测试手段，展现

出在移动应用测试里提高测试效率、保障测试质量、找出潜在稳定性问题的实际用处。经过这些实验和分析，本文对框架的实际应用能力有了直观的了解，给下一章做总结和展望提供了足够的数据支撑和实际经验。



## 第六章 总结与展望

本章对本文的研究工作进行整体回顾与总结，并结合实验结果对所设计的移动应用自动化测试框架进行评价，同时提出后续可能的改进方向和研究展望。通过前五章的研究内容和实验验证，可以看出，自动化测试在提升测试效率、保证功能正确性、记录测试过程以及适应多设备环境等方面具有一定应用价值。同时，本文在固定流程自动化测试之外，补充了 ADB Monkey 随机稳定性测试，使测试方法更加完整，也为分析应用在随机操作场景下的稳定性提供了参考。

### 6.1 研究总结

本文围绕“手机 APP 自动化测试”这个主题展开研究，目标是设计出一套结构清楚、模块化、能重复用还方便维护的移动应用自动化测试框架，以此提高测试效率，同时验证移动应用核心功能和界面交互有没有做对。研究先介绍了手机 APP 测试、自动化测试技术、Appium 与 Selenium 工具、Android 测试环境还有自动化测试框架设计的相关理论，给后面搭框架和做实验测试打下了基础。

在梳理测试需求的阶段，本文围绕 ApiDemos 6.0.10 的功能特点，把功能测试、UI 测试、兼容性、性能测试还有随机稳定性测试这些需求都整理清楚，再根据整理好的需求设计了对应的测试用例。测试用例覆盖了登录、表单、页面跳转、元素可见性、滑动滚动、不同系统版本兼容还有 Monkey 随机稳定性测试等内容，给后面写测试脚本和分析结果给出了明确依据。

在框架设计这块，本文结合 Appium、Selenium WebDriver、UiAutomator2、Page Object 模式、数据驱动思路还有日志报告机制，设计出一个有五层结构的手机 APP 自动化测试框架，这个框架主要分成测试用例管理层、页面元素封装层、公共方法层、配置管理层还有日志与报告层。测试用例管理层用来整理安排测试用例，页面元素封装层用来把元素定位和页面操作打包封装；公共方法层负责处理等待、滑动、异常处理这类通用操作，配置管理层用来管理设备和测试环

境的参数，日志与报告层则用来记录测试的执行过程和最终结果，这样设计结构，一定程度上能提高测试脚本的复用性、稳定性还有可维护性。

在测试实现和分析结果阶段，本文先把 ApiDemos 6.0.10 当成主要测试对象，在 Android 16 模拟器和 Android 13 真机这两个环境里，执行了登录测试、表单测试、元素可见性测试还有 ActivityTest 综合测试。测试过程里，脚本可以完成点击、输入、滑动、页面跳转和结果验证这些操作，大部分测试用例都能按照预期跑通，同时本文还加入 SauceLabs My Demo App 作为补充的测试对象，对购物流程、登录和 Website 搜索这些场景做了测试，这样就不会让测试内容只局限在 ApiDemos 这一个应用上。

另外，本文还补充了利用 ADB Monkey 做的随机稳定性测试，这个测试在 Android 16 模拟器环境里往 ApiDemos 发送了大量随机事件，通过 Monkey 自带的日志信息查看应用会不会出现崩溃、没响应或者异常退出这类问题。测试结果表明，有部分测试轮次可以完整跑完 10000 个随机事件，说明应用在普通随机操作下有一定稳定性，同时，还有一轮测试在 AudioFxDemo 页面触发了 RuntimeException，说明 Monkey 测试能找出固定流程测试不容易覆盖到的边缘页面和藏着的稳定性问题。

总体而言，本文基本走完了从理论分析、需求设计、框架搭建、脚本实现到测试结果分析的完整研究流程。实验结果表明，基于 Appium 和 Selenium 的自动化测试方法可以有效完成手机 APP 的核心功能和界面交互测试；日志记录和测试报告能够帮忙定位出错原因，手动测试跟自动化测试做对比也能看出，自动化测试在重复执行和记录结果方面有着一定优势，同时，加上了 ADB Monkey 随机测试之后，也进一步丰富了本文的测试方法，让研究内容更完整。

## 6.2 研究不足

虽然本文围绕手机 APP 自动化测试框架的设计和实现做了相关研究，还通过多个测试用例还有测试方法对框架做了验证，但受研究时长、实验条件还有个人技术水平这些因素限制，还是存在一些不足。

第一，本次用到的测试对象数量不多，本文主要把 ApiDemos 6.0.10 当成核心测试对象，另外补充了 SauceLabs My Demo App 来做部分场景的测试。虽然

这两个应用能覆盖登录、表单、页面跳转、UI 元素交互还有购物流程这些常见场景，但跟真实的商业 APP 比起来，它们的业务逻辑还是偏简单，还没有覆盖到支付、消息推送、第三方登录还有复杂网络请求这类场景。

第二，测试设备和系统环境覆盖得还不够全面。本文主要是在 Android 16 模拟器和 Android 13 真机上做的实验，能初步验证测试脚本在不同环境下的运行状态，但用到的设备品牌、系统版本、屏幕尺寸还有硬件配置数量仍然偏少，所以这个框架在更复杂设备环境里的兼容性还有适配能力，还需要做进一步验证。

第三，针对性能的测试分析做得不够深入。本文主要靠执行耗时、页面响应和操作流畅度来观察测试结果，没有进一步结合 CPU 使用率、内存占用、电量消耗还有网络延迟这些指标来做分析，所以对框架性能表现的评价还停留在比较基础的层面。

另外，本文虽然加入了 ADB Monkey 随机稳定性测试，但这个测试大多是在 Android 16 模拟器里运行的，没有在真实设备上做大规模执行。这么做是为了让测试环境保持安全可控，但真实设备上的随机稳定性测试，还是有进一步拓展的空间。

最后，本文的自动化测试框架仍有继续完善的地方。虽然框架已经体现了模块化设计、Page Object 思想、日志记录和测试报告机制，但失败重试、显式等待、测试数据管理、报告可视化以及持续集成流程仍可以继续优化。

总体来看，本文完成了手机 APP 自动化测试框架的基本设计与实验验证，但在测试对象规模、设备覆盖范围、性能指标深度、真实设备随机测试和框架工程化完善方面仍存在不足。

## 6.3 后续改进与展望

针对本文研究里存在的不足，后续可以从下面几个方面来改进和拓展。

第一，能进一步扩大测试对象的覆盖范围。除了 ApiDemos 6.0.10 和 Sauce-Labs My Demo App 之外，后续可以挑选更多真实商业 APP 或是开源移动应用来做测试，覆盖登录认证、网络请求、文件操作、支付流程这些不同功能，以此验证这个框架在复杂业务环境里是不是适用。

第二，可以增加测试设备以及系统环境的类型，包括不同 Android 版本、不

同手机品牌、不同屏幕尺寸还有不一样的硬件配置，再结合弱网或者网络切换的场景，来全面观察测试脚本在多设备、多环境下的兼容性和稳定性。

第三，可以把性能测试内容做深化，在现有统计执行耗时的基础上，加入 CPU 使用率、内存占用、电量消耗还有网络延迟这类指标，让性能分析变得更具体也更量化。

另外，还可以完善 ADB Monkey 随机稳定性测试，目前这项测试主要是在 Android 16 模拟器上执行的。后续可以结合更多设备或者隔离环境来展开，把日志、Crash 信息和系统日志分析结合到一起，提高异常定位的能力。

可以对自动化测试框架做优化，比如完善显式等待、失败重试、异常截图、数据管理和报告可视化这些功能，还可以尝试接入 CI/CD 持续集成，做到测试自动触发和执行，提高工程化水平还有应用价值。

总体而言，移动 APP 自动化测试还有很大的研究空间，后续工作可以在测试覆盖、执行效率、稳定性分析还有持续集成这些方面不断完善，为移动应用的质量保障提供更强有力的支持。

## 参考文献

- [1] KONG P, LI L, GAO J, et al. Automated Testing of Android Apps: A Systematic Literature Review[J]. IEEE Transactions on Reliability, 2019, 68(1).
- [2] KIRUBAKARAN B, KARTHIKEYANI V. Mobile application testing —Challenges and solution approach through automation[C]//2013 International Conference on Pattern Recognition, Informatics and Mobile Engineering. 2013: 79-84. DOI: 10.1109/ICPRIME.2013.6496451.
- [3] TAO C, GAO J. On Building Test Automation System for Mobile Applications Using GUI Ripping[C]// . DOI: 10.18293/seke2016-168.
- [4] PRATHIBHAN C M, MALINI A, VENKATESH N, et al. An Automated Testing Framework for Testing Android Mobile Applications in the Cloud[C]//2014 IEEE International Conference on Advanced Communication Control and Computing Technologies (ICACCCT). 2014.
- [5] SINCLAIR C. The Role of Selenium in Mobile Application Testing[J]. International Journal of Advanced and Innovative Research, 2023, 12(1).
- [6] 张佳慧. 基于 Appium 的移动设备自动化软件测试的研究与实现[J]. 信息与电脑 (理论版), 2022, 34(02): 102-107.
- [7] 黄冬丽, 李月峰, 谢添姣, 等. 基于 Selenium-Appium 的多端登录自动化测试方法研究[J]. 电脑编程技巧与维护, 2025(11): 39-42. DOI: 10.16184/j.cnki.comprg.2025.11.013.
- [8] 李佳. 最强的手机测速 App[J]. 计算机与网络, 2021, 47(05): 32.
- [9] GeeksforGeeks. What is Mobile Application Testing?[EB/OL]. 2026. <https://www.geeksforgeeks.org/software-testing/what-is-mobile-application-testing/>.

- [10] GeeksforGeeks. Automation Testing Software Testing[EB/OL]. <https://www.geeksforgeeks.org/software-testing/automation-testing-software-testing/>.
- [11] BrowserStack. BrowserStack[EB/OL]. <https://www.browserstack.com/>.
- [12] AI L T. TestMu AI[EB/OL]. <https://www.testmuai.com/>.
- [13] Appium. Appium[EB/OL]. <https://appium.io/>.
- [14] RAFI D, MOSES K, PETERSEN K, et al. Benefits and limitations of automated software testing: Systematic literature review and practitioner survey[C]//2012 7th International Workshop on Automation of Software Test, AST 2012 - Proceedings. 2012: 36-42. DOI: 10.1109/IWAST.2012.6228988.
- [15] JOIS N R. Automated Software Testing and Quality Control: A Literature Review[Z]. UIC, CS440 Software Engineering. 2024.
- [16] Selenium. Selenium[EB/OL]. <https://www.selenium.dev/>.
- [17] GeeksforGeeks. Selenium – An Automation Tool[EB/OL]. <https://www.geeksforgeeks.org/software-engineering/software-engineering-selenium-an-automation-tool/>.
- [18] BrowserStack. Testing on Emulators, Simulators and Real Devices[EB/OL]. <https://www.browserstack.com/guide/testing-on-emulators-simulators-real-devices-comparison>.
- [19] A M A H, R K J, MAHESWARI D L. Research on Hybrid Automation Framework for Mobile Application Testing Based on Page Object Model and Appium[J]. International Journal of P2P Network Trends and Technology (IJPTT), 2020, 10(4): 12-15. DOI: 10.14445/22492615/IJPTT-V10I4P402.
- [20] BERIHUN N G, CHERIET M A, FENTA B M. The Applicability of Automated Testing Frameworks for Mobile Application Testing: A Systematic Literature Review[J]. Comput. Sci., 2023, 12(5): 97-108.
- [21] GeeksforGeeks. Data Driven Testing[EB/OL]. <https://www.geeksforgeeks.org/software-testing/data-driven-testing/>.

- [22] ShiftAsia. How to Perform Interruption Testing in Mobile Applications[EB/OL].  
<https://shiftasia.com/column/how-to-perform-interruption-testing-in-mobile-applications/>.

## 致 谢

首先，我要衷心感谢我的导师聂长海教授，在论文选题、研究设计和写作过程中给予我悉心指导和耐心帮助。聂老师不仅在学术上给予我专业指导，也在研究方法和论文逻辑方面提供了宝贵建议，使我能够顺利完成毕业论文。

同时，感谢各位答辩老师在答辩过程中提出的建设性意见，让我对研究方法、实验设计和内容完善有了更清晰的认识。

我还要感谢我的家人和同学，在学习和生活中给予我支持与鼓励，使我能够专心投入到毕业论文的研究工作中。特别感谢薛佳恩同学，在数据分析、论文修改和答辩准备过程中提供了细心的帮助和陪伴。

最后，感谢所有在论文写作与研究过程中给予我指导和支持的人，是你们的帮助让我能够顺利完成本科毕业论文。



## 附录 A 测试报告明细

### 额外测试日志结果

| TestTime | TestName       | StepName                    | Status   | Duration(ms) | ErrorPoint                        | ErrorContent | PassedCases | TotalCases | SuccessRate(%) |
|----------|----------------|-----------------------------|----------|--------------|-----------------------------------|--------------|-------------|------------|----------------|
| #####    | LoginTest      | Click Views                 | PASS     | 1727         |                                   |              | 1           | 1          | 100            |
| #####    | LoginTest      | Click TextFields            | PASS     | 6136         |                                   |              | 2           | 2          | 100            |
| #####    | LoginTest      | Input TextFields            | PASS     | 3906         |                                   |              | 3           | 3          | 100            |
| #####    | RadioGroupTest | Back To Main Page           | PASS     | 2551         |                                   |              | 4           | 4          | 100            |
| #####    | RadioGroupTest | Click Views                 | PASS     | 1836         |                                   |              | 5           | 5          | 100            |
| #####    | RadioGroupTest | Click Radio Group           | PASS     | 8964         |                                   |              | 6           | 6          | 100            |
| #####    | RadioGroupTest | Clear Button                | PASS     | 2489         |                                   |              | 7           | 7          | 100            |
| #####    | RadioGroupTest | Snack Button                | PASS     | 2138         |                                   |              | 8           | 8          | 100            |
| #####    | RadioGroupTest | Breakfast Button            | PASS     | 4009         |                                   |              | 9           | 9          | 100            |
| #####    | RadioGroupTest | Lunch Button                | PASS     | 3992         |                                   |              | 10          | 10         | 100            |
| #####    | RadioGroupTest | Dinner Button               | PASS     | 4017         |                                   |              | 11          | 11         | 100            |
| #####    | RadioGroupTest | Back To Main Page           | PASS     | 2510         |                                   |              | 12          | 12         | 100            |
| #####    | ActivityTest   | Click App                   | PASS     | 1928         |                                   |              | 13          | 13         | 100            |
| #####    | ActivityTest   | Click Activity              | PASS     | 1317         |                                   |              | 14          | 14         | 100            |
| #####    | ActivityTest   | Click Custom Title          | PASS     | 1322         |                                   |              | 15          | 15         | 100            |
| #####    | ActivityTest   | Change Left Title           | PASS     | 2224         |                                   |              | 16          | 16         | 100            |
| #####    | ActivityTest   | Change Right Title          | PASS     | 2079         |                                   |              | 17          | 17         | 100            |
| #####    | ActivityTest   | Back From Custom Title      | PASS     | 4691         |                                   |              | 18          | 18         | 100            |
| #####    | ActivityTest   | Hello World                 | PASS     | 1913         |                                   |              | 19          | 19         | 100            |
| #####    | ActivityTest   | Back From Hello World       | PASS     | 690          |                                   |              | 20          | 20         | 100            |
| #####    | ActivityTest   | Click Receive Result        | PASS     | 5839         |                                   |              | 21          | 21         | 100            |
| #####    | ActivityTest   | Click Get Result For Corky  | PASS     | 1211         |                                   |              | 22          | 22         | 100            |
| #####    | ActivityTest   | Corky Button                | PASS     | 2336         |                                   |              | 23          | 23         | 100            |
| #####    | ActivityTest   | Click Get Result For Violet | PASS     | 1172         |                                   |              | 24          | 24         | 100            |
| #####    | ActivityTest   | Violet Button               | PASS     | 5662         |                                   |              | 25          | 25         | 100            |
| #####    | ActivityTest   | Back To Activity Page       | PASS     | 5240         |                                   |              | 26          | 26         | 100            |
| #####    | ActivityTest   | Click Alert Dialogs         | PASS     | 1828         |                                   |              | 27          | 27         | 100            |
| #####    | ActivityTest   | Click Long Message          | PASS     | 226          |                                   |              | 28          | 28         | 100            |
| #####    | ActivityTest   | Long Message Check          | PASS     | 938          |                                   |              | 29          | 29         | 100            |
| #####    | ActivityTest   | Click OK                    | PASS     | 253          |                                   |              | 30          | 30         | 100            |
| #####    | Summary        | All Test Cases              | FINISHED | 99996        | Total Passed: 30, Total Cases: 30 |              | 30          | 30         | 100            |

图 A-1 综合测试全部通过时的 test\_log(6) 执行结果（真机）

| TestTime | TestName       | StepName                    | Status   | Duration(ms) | ErrorPoint                        | ErrorContent | PassedCases | TotalCases | SuccessRate(%) |
|----------|----------------|-----------------------------|----------|--------------|-----------------------------------|--------------|-------------|------------|----------------|
| #####    | LoginTest      | Click Views                 | PASS     | 1787         |                                   |              | 1           | 1          | 100            |
| #####    | LoginTest      | Click TextFields            | PASS     | 6120         |                                   |              | 2           | 2          | 100            |
| #####    | LoginTest      | Input TextFields            | PASS     | 2766         |                                   |              | 3           | 3          | 100            |
| #####    | RadioGroupTest | Back To Main Page           | PASS     | 2216         |                                   |              | 4           | 4          | 100            |
| #####    | RadioGroupTest | Click Views                 | PASS     | 1889         |                                   |              | 5           | 5          | 100            |
| #####    | RadioGroupTest | Click Radio Group           | PASS     | 4574         |                                   |              | 6           | 6          | 100            |
| #####    | RadioGroupTest | Clear Button                | PASS     | 749          |                                   |              | 7           | 7          | 100            |
| #####    | RadioGroupTest | Snack Button                | PASS     | 766          |                                   |              | 8           | 8          | 100            |
| #####    | RadioGroupTest | Breakfast Button            | PASS     | 784          |                                   |              | 9           | 9          | 100            |
| #####    | RadioGroupTest | Lunch Button                | PASS     | 747          |                                   |              | 10          | 10         | 100            |
| #####    | RadioGroupTest | Dinner Button               | PASS     | 735          |                                   |              | 11          | 11         | 100            |
| #####    | RadioGroupTest | Back To Main Page           | PASS     | 1798         |                                   |              | 12          | 12         | 100            |
| #####    | ActivityTest   | Click App                   | PASS     | 560          |                                   |              | 13          | 13         | 100            |
| #####    | ActivityTest   | Click Activity              | PASS     | 2692         |                                   |              | 14          | 14         | 100            |
| #####    | ActivityTest   | Click Custom Title          | PASS     | 1362         |                                   |              | 15          | 15         | 100            |
| #####    | ActivityTest   | Change Left Title           | PASS     | 1780         |                                   |              | 16          | 16         | 100            |
| #####    | ActivityTest   | Change Right Title          | PASS     | 1762         |                                   |              | 17          | 17         | 100            |
| #####    | ActivityTest   | Back From Custom Title      | PASS     | 674          |                                   |              | 18          | 18         | 100            |
| #####    | ActivityTest   | Hello World                 | PASS     | 1852         |                                   |              | 19          | 19         | 100            |
| #####    | ActivityTest   | Back From Hello World       | PASS     | 626          |                                   |              | 20          | 20         | 100            |
| #####    | ActivityTest   | Click Receive Result        | PASS     | 1817         |                                   |              | 21          | 21         | 100            |
| #####    | ActivityTest   | Click Get Result For Corky  | PASS     | 2294         |                                   |              | 22          | 22         | 100            |
| #####    | ActivityTest   | Corky Button                | PASS     | 1215         |                                   |              | 23          | 23         | 100            |
| #####    | ActivityTest   | Click Get Result For Violet | PASS     | 1109         |                                   |              | 24          | 24         | 100            |
| #####    | ActivityTest   | Violet Button               | PASS     | 1382         |                                   |              | 25          | 25         | 100            |
| #####    | ActivityTest   | Back To Activity Page       | PASS     | 1746         |                                   |              | 26          | 26         | 100            |
| #####    | ActivityTest   | Click Alert Dialogs         | PASS     | 1801         |                                   |              | 27          | 27         | 100            |
| #####    | ActivityTest   | Click Long Message          | PASS     | 109          |                                   |              | 28          | 28         | 100            |
| #####    | ActivityTest   | Long Message Check          | PASS     | 963          |                                   |              | 29          | 29         | 100            |
| #####    | ActivityTest   | Click OK                    | PASS     | 100          |                                   |              | 30          | 30         | 100            |
| #####    | Summary        | All Test Cases              | FINISHED | 65885        | Total Passed: 30, Total Cases: 30 |              | 30          | 30         | 100            |

图 A-2 综合测试全部通过时的 test\_log(5) 执行结果（模拟器）

| TestTime | TestName | StepName               | Status   | Duration(ms) | ErrorPoint | ErrorContent                    | PassedCases | TotalCases | SuccessRate(%) |
|----------|----------|------------------------|----------|--------------|------------|---------------------------------|-------------|------------|----------------|
| #####    | Backpack | Click Backpack         | PASS     | 432          |            |                                 | 1           | 1          | 100            |
| #####    | Rating   | Click Rating           | PASS     | 2428         |            |                                 | 2           | 2          | 100            |
| #####    | Add      | Click Add              | PASS     | 966          |            |                                 | 3           | 3          | 100            |
| #####    | Cart     | Click Cart             | PASS     | 1676         |            |                                 | 4           | 4          | 100            |
| #####    | Login    | Click Login            | PASS     | 5576         |            |                                 | 5           | 5          | 100            |
| #####    | Shipping | Input Shipping Address | PASS     | 7196         |            |                                 | 6           | 6          | 100            |
| #####    | Payment  | Input Payment          | PASS     | 21189        |            |                                 | 7           | 7          | 100            |
| #####    | WebView  | Click WebView          | PASS     | 36841        |            |                                 | 8           | 8          | 100            |
| #####    | Summary  | All Test Cases         | FINISHED | 97849        |            | Total Passed: 8, Total Cases: 8 | 8           | 8          | 100            |

图 A-3 My Demo App 通过全部测试集的执行结果（真机）

| TestTime | TestName | StepName               | Status   | Duration(ms) | ErrorPoint | ErrorContent                    | PassedCases | TotalCases | SuccessRate(%) |
|----------|----------|------------------------|----------|--------------|------------|---------------------------------|-------------|------------|----------------|
| #####    | Backpack | Click Backpack         | PASS     | 1827         |            |                                 | 1           | 1          | 100            |
| #####    | Rating   | Click Rating           | PASS     | 1505         |            |                                 | 2           | 2          | 100            |
| #####    | Add      | Click Add              | PASS     | 2156         |            |                                 | 3           | 3          | 100            |
| #####    | Cart     | Click Cart             | PASS     | 1643         |            |                                 | 4           | 4          | 100            |
| #####    | Login    | Click Login            | PASS     | 6660         |            |                                 | 5           | 5          | 100            |
| #####    | Shipping | Input Shipping Address | PASS     | 7678         |            |                                 | 6           | 6          | 100            |
| #####    | Payment  | Input Payment          | PASS     | 12997        |            |                                 | 7           | 7          | 100            |
| #####    | WebView  | Click WebView          | PASS     | 37957        |            |                                 | 8           | 8          | 100            |
| #####    | Summary  | All Test Cases         | FINISHED | 90942        |            | Total Passed: 8, Total Cases: 8 | 8           | 8          | 100            |

图 A-4 My Demo App 通过全部测试集的执行结果（模拟器）

## 附录 B 本文代码存储库

<https://github.com/linnengyuu/NJU-Thesis>